ETUDE SUR LA FAIM DANS LE MONDE

I - Analyse exploratoire des fichiers

1.1 - Importation des librairies

In [3]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

1.2 - Analyse exploratoire du fichier population

a) Chargement et description des tables

In [5]:
```
# Importation du fichier population.csv
pop = pd.read_csv("population.csv")

# Exclusion des informations de l'année 2018 car l'analyse porte sur la période 201
pop = pop.loc[pop["Année"] != 2018, :]

# Affichage des dimensions de la table population
pop.shape
```

Out[5]:
```
(1180, 3)
```

In [152…
```
# Affichage des caractéristiques de la table population
pop.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 1180 entries, 0 to 1414
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Zone    1180 non-null   object
 1   Année   1180 non-null   int64
 2   Valeur  1180 non-null   float64
dtypes: float64(1), int64(1), object(1)
memory usage: 36.9+ KB
```

In [9]:
```
# Impression du nombre de modalités de la variable zone

print("La variable Zone est composée de",pop.groupby("Zone")["Zone"].count().shape[
      "modalités correpondant aux différentes zones géographiques prises en compte
```

```
La variable Zone est composée de 236 modalités correpondant aux différentes zones gé
ographiques prises en compte dans l'étude.
```

b) Vérification de la complétude des données

Chaque zone est observée sur la période 2013 à 2017. Ducoup chaque zone est supposée avoir 5 observations annuelles.

In [11]:
```
# Identificaton des zones ayant moins de 5 observations annuelles

df = pop.groupby("Zone")["Année"].count()
df = df.reset_index()
print("Le nombre de zones ayant moins de cinq observations annuelles est de ",
      df.loc[df["Année"] < 5, :].shape[0])
```

```
Le nombre de zones ayant moins de cinq observations annuelles est de  0
```

In [13]:
```
# Affichage des 5 premières lignes de la table
pop.head()
```

Out[13]:

	Zone	Année	Valeur
0	Afghanistan	2013	32269.589
1	Afghanistan	2014	33370.794
2	Afghanistan	2015	34413.603
3	Afghanistan	2016	35383.032
4	Afghanistan	2017	36296.113

c) Standardisation et renommage des variables

In [15]:
```
# Nous allons harmoniser les unités. Pour cela, nous avons décidé de multiplier la
# Multiplication de la colonne valeur par 1000 pour avoir la taille de la populatio
pop["Valeur"] = pop["Valeur"] * 1000

# changement du nom de la colonne Valeur par Population
pop.rename(columns={"Valeur":"population", "Zone":"zone", "Année":"année"},
           inplace=True)

pop.loc[pop["zone"] == "République populaire démocratique de Corée", "zone"] = "Cor


# sauvegarde de la table au format csv
```

```
pop.to_csv("df_population.csv", index=False)


# Affichage des 5 premières lignes de la table pour voir les modifications
pop.head()
```

Out[15]:

	zone	année	population
0	Afghanistan	2013	32269589.0
1	Afghanistan	2014	33370794.0
2	Afghanistan	2015	34413603.0
3	Afghanistan	2016	35383032.0
4	Afghanistan	2017	36296113.0

1.3 - Analyse exploratoire du fichier disponibilité alimentaire

a) Chargement et description des tables

In [17]:

```
# Importation du fichier dispo_alimentaire.csv
dispo_al = pd.read_csv("dispo_alimentaire.csv")

# Afficher les dimensions de la table sur la disponibilité alimentaire pour l'année
dispo_al.shape
```

Out[17]: (15605, 18)

In [164…

```
# Caractéristique de la table disponibilité alimentaire
dispo_al.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15605 entries, 0 to 15604
Data columns (total 18 columns):
 #   Column                                                      Non-Null Count
Dtype
---  ------                                                      --------------
-----
 0   Zone                                                        15605 non-null
object
 1   Produit                                                     15605 non-null
object
 2   Origine                                                     15605 non-null
object
 3   Aliments pour animaux                                       2720 non-null
float64
 4   Autres Utilisations                                         5496 non-null
float64
 5   Disponibilité alimentaire (Kcal/personne/jour)              14241 non-null
float64
 6   Disponibilité alimentaire en quantité (kg/personne/an)      14015 non-null
float64
 7   Disponibilité de matière grasse en quantité (g/personne/jour)  11794 non-null
float64
 8   Disponibilité de protéines en quantité (g/personne/jour)    11561 non-null
float64
 9   Disponibilité intérieure                                    15382 non-null
float64
 10  Exportations - Quantité                                     12226 non-null
float64
 11  Importations - Quantité                                     14852 non-null
float64
 12  Nourriture                                                  14015 non-null
float64
 13  Pertes                                                      4278 non-null
float64
 14  Production                                                  9180 non-null
float64
 15  Semences                                                    2091 non-null
float64
 16  Traitement                                                  2292 non-null
float64
 17  Variation de stock                                          6776 non-null
float64
dtypes: float64(15), object(3)
memory usage: 2.1+ MB
```

In [165…

```
# Affichage des 5 premieres lignes de la table
dispo_al.head()
```

Out[165…

	Zone	Produit	Origine	Aliments pour animaux	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour)	Dispo alimen c (kg/perso
0	Afghanistan	Abats Comestible	animale	NaN	NaN	5.0	
1	Afghanistan	Agrumes, Autres	vegetale	NaN	NaN	1.0	
2	Afghanistan	Aliments pour enfants	vegetale	NaN	NaN	1.0	
3	Afghanistan	Ananas	vegetale	NaN	NaN	0.0	
4	Afghanistan	Bananes	vegetale	NaN	NaN	4.0	

b) Vérification de la complétude des données

In [19]:
```
# Nombre de zones pour lesquelles on dispose des informations

print("On dispose des informations uniquement pour", dispo_al.groupby("Zone")["Zone
```

```
On dispose des informations uniquement pour 174 zones sur un total de 236.
```

In [21]:
```
# Construction de la liste des zones dont on ne dispose aucune informations sur les

# liste des zones de la table population
liste_zone_pop = df["Zone"].unique().tolist()

# Liste des zones de la table dispo alimentaire
df2 = dispo_al.groupby("Zone")["Origine"].count()
df2 = df2.reset_index()
liste_zone_disp_al = df2["Zone"].unique().tolist()

# liste des zones dont on ne dispose aucune information sur la disponibilité alimen

liste_zone_non_dispo = []

for z in liste_zone_pop:
    if z not in liste_zone_disp_al:
        liste_zone_non_dispo.append(z)
print("Liste des zones dont on ne dispose aucune information sur les disponibilités
      liste_zone_non_dispo)
```

```
Liste des zones dont on ne dispose aucune information sur les disponibilités aliment
aires dans la table disponibilité alimentaire:  ['Andorre', 'Anguilla', 'Antilles né
erlandaises (ex)', 'Aruba', 'Bahreïn', 'Bhoutan', 'Bonaire, Saint-Eustache et Saba',
'Burundi', 'Comores', 'Curaçao', 'Gibraltar', 'Groenland', 'Guadeloupe', 'Guam', 'Gu
inée équatoriale', 'Guyane française', 'Libye', 'Liechtenstein', 'Martinique', 'Mayo
tte', 'Micronésie (États fédérés de)', 'Monaco', 'Montserrat', 'Nauru', 'Nioué', 'Pa
laos', 'Palestine', 'Papouasie-Nouvelle-Guinée', 'Porto Rico', 'Qatar', "Royaume-Uni
de Grande-Bretagne et d'Irlande du Nord", 'République arabe syrienne', 'République d
émocratique du Congo', 'Réunion', 'Sahara occidental', 'Saint-Barthélemy', 'Saint-Ma
rin', 'Saint-Martin (partie française)', 'Saint-Pierre-et-Miquelon', 'Saint-Siège',
'Sainte-Hélène, Ascension et Tristan da Cunha', 'Samoa américaines', 'Seychelles',
'Singapour', 'Sint Maarten  (partie néerlandaise)', 'Somalie', 'Soudan du Sud', 'Tch
équie', 'Tokélaou', 'Tonga', 'Tuvalu', 'Érythrée', 'Île de Man', 'Îles Anglo-Normand
es', 'Îles Caïmanes', 'Îles Cook', 'Îles Falkland (Malvinas)', 'Îles Féroé', 'Îles M
ariannes du Nord', 'Îles Marshall', 'Îles Turques-et-Caïques', 'Îles Vierges américa
ines', 'Îles Vierges britanniques', 'Îles Wallis-et-Futuna']
```

Les informations sur la disponibilité alimentaire de ces zones ont été receullies dans le site de la FAO. Ainsi une table avec les disponbilités alimentaires de l'année 2017 dans ces zones a été construite

In [23]:
```
# Construction de la table sur les disponibilités alimentaires des zones su évoquée

# Chargement des tables et selection des variables necessaires
dispo_alim = pd.read_csv("FAOSTAT_data_fr_5-21-2025.csv")
dispo_alim = dispo_alim.drop(columns = ["Code Domaine", "Domaine", "Code année", "C
                            "Description du Symbole", "Note", "Code Élément"])

# Réorganisation de la table
dispo_alim_2 = dispo_alim.pivot_table(index=["Zone", "Produit", "Unité"],  columns=
dispo_alim_2 = dispo_alim_2.reset_index()
```

In [25]:
```
# Impression de la table

dispo_alim_2.head()
```

Out[25]:

	Élément	Zone	Produit	Unité	Aliments pour animaux	Autres utilisations (non alimentaire)	Disponibili alimentai (Kcal/personne/jou
0		Burundi	Abats et foies comestibles d'oie, frais, réfri...	g/personne/jour	NaN	NaN	Na
1		Burundi	Abats et foies comestibles d'oie, frais, réfri...	kcal/personne/jour	NaN	NaN	(
2		Burundi	Abats et foies comestibles d'oie, frais, réfri...	tonnes	NaN	NaN	Na
3		Burundi	Abats et foies comestibles de canard, frais, r...	g/personne/jour	NaN	NaN	Na
4		Burundi	Abats et foies comestibles de canard, frais, r...	kcal/personne/jour	NaN	NaN	(

In [27]:

```
print("Cette nouvelle table contient jusqu'à ", len(dispo_alim_2["Produit"].unique(
      "produits différents.", end = " ")
print("Alors que la table de disponibilité alimentaire initiale contient seulement"
      len(dispo_al["Produit"].unique().tolist()), "produits.")
```

```
Cette nouvelle table contient jusqu'à  431 produits différents. Alors que la table d
e disponibilité alimentaire initiale contient seulement 98 produits.
```

In [29]:

```
# Produits de la table disponibilité alimentaire initiale
print("Liste des produits de la table disonibilité alimentaire initiale : ",
      dispo_al["Produit"].unique().tolist())
print()
# Produits de la 2e table de disponibilité alimentaire
print("Liste des produits de la 2e table:", dispo_alim_2["Produit"].unique().tolist
```

Liste des produits de la table disonibilité alimentaire initiale : ['Abats Comestible', 'Agrumes, Autres', 'Aliments pour enfants', 'Ananas', 'Bananes', 'Beurre, Ghee', 'Bière', 'Blé', 'Boissons Alcooliques', 'Café', 'Coco (Incl Coprah)', 'Crème', 'Céréales, Autres', 'Dattes', 'Edulcorants Autres', 'Feve de Cacao', 'Fruits, Autres', 'Graines de coton', 'Graines de tournesol', 'Graisses Animales Crue', 'Huil Plantes Oleif Autr', 'Huile Graines de Coton', "Huile d'Arachide", "Huile d'Olive", 'Huile de Colza&Moutarde', 'Huile de Palme', 'Huile de Soja', 'Huile de Sésame', 'Huile de Tournesol', 'Lait - Excl Beurre', 'Légumes, Autres', 'Légumineuses Autres', 'Maïs', 'Miel', 'Millet', 'Miscellanees', 'Noix', 'Oeufs', 'Olives', 'Oranges, Mandarines', 'Orge', 'Plantes Oleiferes, Autre', 'Poissons Eau Douce', 'Poivre', 'Pommes', 'Pommes de Terre', 'Raisin', 'Riz (Eq Blanchi)', 'Sucre Eq Brut', 'Sucre, betterave', 'Sucre, canne', 'Sésame', 'Thé', 'Tomates', "Viande d'Ovins/Caprins", 'Viande de Bovins', 'Viande de Volailles', 'Viande, Autre', 'Vin', 'Épices, Autres', 'Alcool, non Comestible', 'Animaux Aquatiques Autre', 'Arachides Decortiquees', 'Avoine', 'Bananes plantains', 'Boissons Fermentés', 'Cephalopodes', 'Citrons & Limes', 'Crustacés', 'Girofles', 'Graines Colza/Moutarde', 'Haricots', 'Huile de Coco', 'Huile de Germe de Maïs', 'Huile de Palmistes', 'Huiles de Foie de Poisso', 'Huiles de Poissons', 'Ignames', 'Manioc', 'Mollusques, Autres', 'Oignons', 'Palmistes', 'Pamplemousse', 'Patates douces', 'Perciform', 'Piments', 'Plantes Aquatiques', 'Pois', 'Poissons Marins, Autres', 'Poissons Pelagiques', 'Racines nda', 'Seigle', 'Soja', 'Sorgho', 'Viande de Suides', 'Huile de Son de Riz', 'Sucre non centrifugé', 'Viande de Anim Aquatiq']

Liste des produits de la 2e table: ["Abats et foies comestibles d'oie, frais, réfrigérés ou congelés", 'Abats et foies comestibles de canard, frais, réfrigérés ou congelés', 'Abats, comestibles, bovins, frais, réfrigérés ou congelés', 'Abats, comestibles, caprins, frais, réfrigérés ou congelés', 'Abats, comestibles, ovins, frais, réfrigérés ou congelés', 'Abats, comestibles, suides, frais, réfrigérés ou congelés', 'Abricots', 'Acides gras monocarboxyliques industriels; huiles acides de raffinage', 'Ail frais', "Alcool éthylique non dénaturé d'un titre alcoométrique volumique de 80 \xa0% vol ou plus", "Alcool éthylique non dénaturé d'un titre alcoométrique volumique de moins de 80\xa0% vol; eaux-de-vie, liqueurs et autres boissons spiritueuses", 'Aliments pour enfants', 'Aliments, préparations de farines, semoules ou extraits de malt', 'Aliments, préparations nda', 'Amandes non décortiquées', 'Amandes, écalées', 'Amidon de maïs', 'Amidons et fécules de blé', 'Ananas', 'Anis, badiane, coriandre, cumin, carvi, fenouil et baies de genièvre, crus', 'Arachides décortiquées', 'Arachides non décortiquées', 'Arachides préparées', 'Asperges', 'Autres boissons caloriques sans alcool', 'Autres espèces cultivées stimulantes, à épices et aromatiques, n.c.a.', 'Autres fruits n.a.c.', 'Autres fruits secs n.a.c.', 'Autres fruits à coques (à l’exclusion des fruits à coque et arachides que l’on trouve dans la nature), en coque, n.c.a.', 'Autres graines oleagineuses n.a.c.', 'Autres haricots frais', 'Autres huiles d’origine végétale, brutes', 'Autres légumes frais n.a.c.', 'Autres légumes, provisoirement conservés', 'Autres légumineuses n.a.c.', 'Autres viandes et abats comestibles, salés, en saumure, séchés ou fumés; farines et poudres, comestibles, de viandes ou d’abats', 'Avocats', 'Avoine', 'Avoine laminée', 'Babeurre, lait caillé, lait acidifié', 'Bananes', 'Beurre de lait de vache', 'Beurre, graisse et huile de cacao', 'Bière de mil, maltée', 'Bière de sorgho, maltée', 'Bière d’orge, maltée', 'Blé', 'Cacao, poudre et masse', 'Café, décaféiné ou torréfié', 'Café, extraits', 'Café, succédanés', 'Café, vert', 'Cannelle et fleurs de cannelier, crues', 'Carottes et navets', 'Caséine', 'Ceréales n.a.c.', 'Champignons en boîte', 'Champignons et truffes', 'Champignons secs', 'Choux', 'Cidre et autres boissons fermentées', 'Citrons et limes', 'Clous de girofles (tiges entières), crus', 'Crème fraiche', 'Crème glacée et autres glaces de consommation', 'Céréales pour petit déjeuner', 'Céréales, préparations', 'Dattes', 'Dégras', 'Extraits, essences et concentrés de café, de thé ou de maté et préparations à base de ces produits ou à base de thé ou maté', 'Farine de blé et de méteil', 'Farine de céréales n.a.c.', 'Farine de fruits', 'Farine de légumin

euses', 'Farine de manioc', 'Farine de maïs', 'Farine de mil', 'Farine de moutarde', 'Farine de sorgho', 'Farine, semoule, poudre, flocons, granulés et agglomérés de pommes de terre', 'Farines de graines ou de fruits oléagineux, autres que la farine de moutarde', 'Feuilles de maté', 'Feuilles de thé)', 'Figues', 'Fromage fondu', 'Fromage, au lait de vache entier', 'Fromage, au lait de vache écrémé', 'Fructoses et sirops, autres', 'Fruits cuits homogénéisés, préparés', 'Fruits préparés nda', 'Fruits à coque préparées', 'Fruits, noix, écorces confits', 'Fèves de soja', 'Fèves et féverole, vertes', 'Fécule de pommes de terres', 'Ghee de lait de vache', 'Gingembre, brut', 'Glucose et dextrose', 'Gluten de blé', 'Graines de coton', 'Graines de lin', 'Graines de ricin', 'Graines de tournesol', 'Graisse de bovins, non fondue', 'Graisse de chèvre, non fondue', 'Graisse de mouton, non fondue', 'Graisses de porcs', 'Graisses et huiles animales ou végétales et leurs fractions, chimiquement modifiées, à l’excl. de celles qui ont été hydrogénées, interestérifiées, réestérifiées ou élaïdinisées; mélanges ou prép. non aliment. de graisses ou d’huiles animales ou végétales', 'Graisses nda, préparations', 'Haricots secs', 'Hosties, cachets vides des types utilisés pour un usage pharmaceutique, pâtes séchées de farine, d’amidon ou de fécule en feuilles et produits similaires.', "Huile d'arachide", "Huile d'olives", 'Huile de coco', 'Huile de colza ou de canola, brute', 'Huile de coton', 'Huile de graines de sésame', 'Huile de graines de tournesol, brute', "Huile de grignons d'olive", 'Huile de lin', 'Huile de maïs', 'Huile de palme', 'Huile de palmiste', 'Huile de ricin', 'Huile de soja', "Huile et graisse d'origine animale n.a.c.", 'Huile et graisse végétale hydrogénée', 'Huile, noix de palme', 'Ignames', "Jus d'agrumes n.a.c.", "Jus d'agrumes n.a.c., concentrés", "Jus d'ananas", "Jus d'ananas concentré", "Jus d'orange", "Jus d'orange concentré", 'Jus de fruits n.a.c.', 'Jus de pamplemousse', 'Jus de pamplemousse concentré', 'Jus de pomme', 'Jus de pomme concentré', 'Jus de raisin', 'Kiwis', 'Lactose', 'Lait cru de brebis', 'Lait cru de chèvre', 'Lait cru de vache', 'Lait, babeurre en poudre', 'Lait, entier concentré non sucré', 'Lait, entier concentré sucré', 'Lait, entier en poudre de vache', 'Lait, écrémé concentré non sucré', 'Lait, écrémé concentré sucré', 'Lait, écrémé de vache', 'Lait, écrémé en poudre de vache', 'Laitue et chicorée', 'Lentilles secs', 'Légumes conservés (surgelés)', 'Légumes conservés n.d.a. (autrement que dans le vinaigre)', 'Légumes déshydratés', 'Légumes surgelés', 'Légumes, légumineuses et pommes de terre, conservées dans le vinaigre ou l’acide acétique', 'Légumes, préparation de homogénéisés', 'Malt, extrait', 'Malt, même torréfié', 'Mangues, mangoustans et goyaves', 'Manioc séché', 'Manioc, frais', 'Margarine et graisses alimentaire compactes', 'Maïs', 'Maïs doux, congelé', 'Maïs doux, préparé ou en conserve', 'Miel naturel', 'Mils', 'Moutarde', 'Mélanges et pâtes pour la préparation des produits de la boulangerie, de la pâtisserie ou de la biscuiterie', 'Mélasses', 'Noix de coco', 'Noix de coco desséchées', 'Noix de muscade, fleur de muscade, cardamomes, crues', 'Noix du brésil, écalées', 'Noix, écalées', 'Oignons et échalotes, séchés (à l’exclusion des formes déshydratées)', 'Olives', 'Olives en conserve', 'Oranges', 'Ovalbumine', 'Pain', 'Palmistes', 'Pastèques', 'Patates douces', 'Piments forts et piments, secs (Capsicum spp., Pimenta spp.), crus', 'Piments forts et piments, verts (Capsicum spp., Pimenta spp.)', 'Pistaches non décortiquées', 'Plantains et bananes à cuire', 'Poires', 'Pois cajan, sec', 'Pois chiches, secs', 'Pois frais', 'Pois secs', 'Poivre (sous-espèces piper), brut', 'Pommes', 'Pommes de terre', 'Pommes de terre surgelées', 'Produits cacaotés nda', 'Produits laitiers n.c.a.', 'Prunes et prunelles', 'Prunes séchées', 'Préparations de foie', 'Préparations à base de viande de volaille', 'Pâte de cacao non dégraissée', 'Pâtes alimentaires, non cuites ni farcies ni autrement préparées', 'Pâtisseries', 'Pêches et nectarines', 'Racines et tubercules similaires à haute teneur en fécule ou en inuline, n.c.a., fraîches', 'Raisin, moût', 'Raisins', 'Raisins secs', 'Riz', 'Riz décortiqué', 'Riz, blanchi', 'Riz, brisures', 'Saucisses et produits similaires, de viande, d’abats ou de sang de porc', 'Seigle', 'Soja, sauce', 'Son de blé', 'Son de céréales n.a.c.', 'Son de maïs', 'Son de mil', 'Son de riz', 'Son de sorgho', 'Sorgho', 'Stéarine solaire et huile de saindoux', 'Sucre de betterave ou de canne brut (centrifugé uniquement)', 'Sucre et sirops n.c.a.', 'Sucre raffiné', 'Sucre, canne',

'Sucre, confiseries', "Sucres raffinés de canne ou de betterave, à l'état solide, ad
ditionnés d'aromatisants ou de colorants; sucre et sirop d'érable", 'Suif', 'Suint d
e laine et lanoline', 'Sérum, en poudre', 'Sérum, fromage', 'Sésame', 'Tangerines, m
andarines, clémentines', 'Taro', 'Tomates, fraiches', 'Tomates, pelées (autrement qu
e dans le vinaigre)', 'Tomates, purée', 'Tourteau de tournesol', "Tourteaux d'arachi
de", 'Tourteaux de colza', 'Tourteaux de coton', 'Tourteaux de graines oléagineuses
nda', 'Tourteaux de palmiste', 'Tourteaux de soja', 'Vanille, brut', 'Vermouths et a
utres vins de raisins frais préparés à l'aide de plantes ou de substances aromatique
s', 'Viande de porc, en morceaux, salée, séchée ou fumée (jambon et lard)', 'Viande
nda, préparées', 'Viande ovine, fraîche ou réfrigérée', 'Viande, boeuf et veau nda,
préparations', 'Viande, boeuf séchée, salée, fumée', 'Viande, bovin, désossée, fraîc
he ou réfrigérée', 'Viande, bovine, fraîche ou réfrigérée', 'Viande, canard, fraîche
ou réfrigérée', 'Viande, caprin, fraîche ou réfrigérée', 'Viande, dinde, fraîche ou
réfrigérée', 'Viande, leporidés, fraîche ou réfrigérée', 'Viande, oie, fraîche ou ré
frigérée', 'Viande, porc, fraîche ou réfrigérée', 'Viande, porc, préparations', 'Via
nde, poulet, fraîche ou réfrigérée', 'Viande, préparations homogénéisée', 'Viande, s
uidés, fraîche ou réfrigérée', 'Viandes nda', 'Vin', 'Yaourt, avec additifs', 'Œufs
de poule en coquille frais', 'Œufs, liquides', 'Œufs, poudre', 'Abats et foies comes
tibles de poulets et de pintades, frais, réfrigérés ou congelés', 'Amidons et fécule
s de riz', 'Ananas, autrement préparés ou conservés', 'Autre jus de légumes nda', 'A
utres fruits, agrumes nda', 'Cabosses et pellicules de cacao', 'Cacao, fèves', 'Conc
ombres, cornichons', 'Copra', 'Farine de céréales mélangées', 'Foie gras, préparatio
ns', 'Graisse de porc, fondue', 'Graisses fondues de volaille', 'Huile de graines de
moutarde, brute', 'Jus de citron', 'Noisettes, écalées', 'Noix non décortiquées', 'O
ignons, echalotes, frais', 'Poireaux et autres légumes alliacés', 'Pois à vache sec
s', 'Saucisses et produits similaires, de viande, d'abats ou de sang de bœuf et de v
eau', 'Tapioca de fécule de pommes de terre', 'Tourteaux de copra', 'Viande, buffle,
fraîche ou réfrigérée', 'Abats et foies comestibles de dinde, frais, réfrigérés ou c
ongelés', "Abats n.a.c. (à l'exclusion des mammifères), frais, réfrigérés ou congelé
s", 'Abricots séchés', 'Amidon de manioc', 'Artichauts', 'Aubergines', 'Autres fruit
s tropicaux frais nda', 'Autres fruits tropicaux secs n.a.c.', 'Boissons obtenues pa
r fermentation du riz', 'Cacahuète, beurre', 'Cantaloup et autres melons', 'Choux-fl
eurs et brocolis', 'Châtaignes non décortiquées', 'Epinards', 'Escargots, autres que
de mer, frais, réfrigérés, congelés, séchés, salés ou en saumure', 'Extraits de jus
de viande, de poissons ou de crustacés, de mollusques ou d'autres invertébrés aquati
ques', 'Farine de fonio', 'Farine de racines et tubercules n.a.c.', 'Farine de riz',
'Figues sèches', 'Fonio', 'Fraises', 'Fèves et féveroles, sèches', 'Germe de blé',
'Germe de maïs', 'Graines de melon', 'Graines de navette ou de colza', 'Graisses, ré
sidus de substances', "Gruau d'orge", 'Jus de citron concentré', 'Jus de mangue', 'J
us de tangerine', 'Jus de tomates', 'Légumes, produits vegetaux frais ou secs n.a.
c.', 'Margarine liquide', 'Maïs «vert» (maïs)', 'Noisettes non décortiquées', "Noix
d'acajou non décortiquées", "Noix d'acajou, écalées", "Noix d'arec", 'Noix de kola',
'Oeillette', 'Plaqueminiers/kakis', 'Quinoa', 'Racines de chicorée', 'Son de céréale
s mélangées', 'Son de fonio', 'Sérum, condensé', 'Tapioca de manioc', 'Tourteaux de
lin', 'Viande de gibier, fraîche, réfrigérée ou congelée', 'Yaourt', 'Alpiste', 'Can
neberges', 'Cerises', 'Citrouilles, courges et potirons', 'Farine de seigle', 'Framb
oises', 'Fructose, chimiquement pur', 'Gombo', 'Haricots verts', 'Mangue, pulpe', 'N
oix du brésil non décortiquées', 'Orge', 'Pamplemousses et pomelos', 'Papayes', "Pom
me d'acajou", 'Racines et tubercules similaires à haute teneur en fécule ou en inuli
ne, n.c.a., secs', 'Sucre, betterave', "Œufs d'autres volatiles en coquille frais n.
a.c.", 'Abats comestibles de chameaux et autres camélidés, frais, réfrigérés ou cong
elés', 'Autre fruit à noyau n.a.c.', 'Autres baies et fruits du genre caccinium, n.
a.c.', 'Carthame', 'Céréales mélangées', 'Fromage, au lait de brebis, frais ou dériv
é', 'Graisse de camélidés', 'Lait cru de chamelle', 'Maltose, chimiquement pur', 'Sa
rrasin, blé noir', "Son d'orge", 'Viande, camélidé, fraîche ou réfrigérée', 'Abats c
omestibles de chevaux et autres équidés, frais, réfrigérés ou congelés', 'Beurre de

```
lait de chèvre', 'Cassis et groseilles', 'Grillottes', 'Myrtilles', 'Son de légumine
uses', 'Triticale', 'Viande, cheval, fraîche ou réfrigérée', 'Abats, comestibles, bu
ffle, frais, réfrigérés ou congelés', 'Beurre de lait de bufflesse', 'Beurre et ghee
de brebis', 'Boulgour', 'Caroubes', 'Chanvre', 'Coings', 'Fourrages et tourteaux au
gluten', 'Fromage de lait de bufflonne, frais ou dérivé', 'Fromage de lait de chèvr
e, frais ou dérivé', 'Ghee de lait de bufflesse', 'Gluten de maïs', 'Graisse de buff
le, non fondue', 'Huile de ricin hydrogénée', 'Isoglucose', 'Lait cru de bufflesse',
'Lait, écrémé de brebis', 'Lupins', 'Pois bambara, secs', 'Riz, usiné (décortiqué)',
'Sérum, frais', 'Tourteau de sésame', 'Vesces', 'Viande, pigeons et autres volatiles
n.a.c., fraîche, réfrigérée ou congelée', 'Bière de maïs, maltée', 'Chou caraibe',
'Graisses de volaille', 'Orge perlé', "Son d'avoine", 'Sucre, autres plantes sucrièr
es nda', "Farines et semoules d'orge", 'Gluten de riz', 'Graisses de découpe de bovi
ns', 'Graisses de découpe de porc', 'Groseilles à maquereaux', 'Huile de carthame',
'Son de seigle']
```

Il y'a donc eu un regroupement des produits dans la table de disponibilité alimentaire initiale. Ne sachant pas commment le regroupement a été effectué, il sera donc tres difficile de réaliser le meme regroupement dans la nouvelle table. Par conséquent nous ne pouvons procéder à une complétude d'informations, à travers une jointure entre la nouvelle table de disponibiloité alimentaire et l'ancienne. la suite de l'analyse s'effectuera donc avec la table de disponibilité alimentaire initiale dans laquelle des informations sur la disponibilité alimentaire dans certaines zones sont manquantes.

In [31]:
```
# Calcul de la population mondiale de 2017 , en proportion de la poopulation totale
pop_mondiale = pop.groupby("année")["population"].sum()
pop_mondiale = pop_mondiale.reset_index()

# Population mondiale surlaquelle les informations sur la disponibilité alimentaire
pop_dispo_al = pop.loc[pop["zone"].isin(liste_zone_disp_al), :]
pop_dispo_al = pop_dispo_al.groupby("année")["population"].sum()
pop_dispo_al = pop_dispo_al.reset_index()

# Renommer les colonnes
pop_dispo_al.rename(columns = {"population":"pop-dispo_al"}, inplace=True)

# Création d'une table qui regroupe les deux types de population mondiale
pop_mond_dispo_al = pop_dispo_al
pop_mond_dispo_al["pop_mond_totale"] =  pop_mondiale["population"]


# Création des proportions des population
pop_mond_dispo_al["prop_pop_disp_al"] =  round(pop_mond_dispo_al["pop-dispo_al"]/po
pop_mond_dispo_al["prop_pop_non_disp_al"] = 100 - pop_mond_dispo_al["prop_pop_disp_
pop_mond_dispo_al.head()
```

Out[31]:

	année	pop-dispo_al	pop_mond_totale	prop_pop_disp_al	prop_pop_non_disp_al
0	2013	6.944803e+09	7.210846e+09	96.31	3.69
1	2014	7.025939e+09	7.295558e+09	96.30	3.70
2	2015	7.106663e+09	7.380067e+09	96.30	3.70
3	2016	7.186864e+09	7.464295e+09	96.28	3.72
4	2017	7.266471e+09	7.548134e+09	96.27	3.73

Les zones surlesquelles on ne dispose aucune informations sur les disponibilités alimentaires pour l'année 2017 représentent moins de 4% de la population mondiale. par conséquent ne pas tenir compte de ces zones n'aura tres fort probablement pas d'impact sur les résultats de nos analyses.

In [34]:

```
# Affichage des 5 premières lignes de la table

dispo_al.head()
```

Out[34]:

	Zone	Produit	Origine	Aliments pour animaux	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour)	Dispo alimen c (kg/perso
0	Afghanistan	Abats Comestible	animale	NaN	NaN	5.0	
1	Afghanistan	Agrumes, Autres	vegetale	NaN	NaN	1.0	
2	Afghanistan	Aliments pour enfants	vegetale	NaN	NaN	1.0	
3	Afghanistan	Ananas	vegetale	NaN	NaN	0.0	
4	Afghanistan	Bananes	vegetale	NaN	NaN	4.0	

c) Standardisation et renommage des variables

In [38]:

```
# Création d'un dictionnaire des variables de la table disponibilité alimentaire

dict_variables = {"aliments": "Aliments pour animaux", "autres_ut":"Autres Utilisat
                  "dispo_alim_kg": "Disponibilité alimentaire en quantité (kg/perso
                  "dispo_proteine": "Disponibilité de protéines en quantité (g/pers
                  "exportations": "Exportations - Quantité", "importations": "Impor


for i in dict_variables:
  print(i, ":", dict_variables[i])
```

```
aliments : Aliments pour animaux
autres_ut : Autres Utilisations
dispo_alim_kcal : Disponibilité alimentaire (Kcal/personne/jour)
dispo_alim_kg : Disponibilité alimentaire en quantité (kg/personne/an)
dispo_matiere_gr : Disponibilité de matière grasse en quantité (g/personne/jour)
dispo_proteine : Disponibilité de protéines en quantité (g/personne/jour)
dispo_int : Disponibilité intérieure = production  + importations + variation des st
ocks
exportations : Exportations - Quantité
importations : Importations - Quantité
variation_stock : Variation de stock
 :
```

In [40]:

```python
# Définition d'une liste des noms des variables de la table disponibilité alimentai
variables_dispo_al = dispo_al.columns.tolist()

# Remplacement des NaN dans le dataset par des 0
dispo_al[variables_dispo_al] = dispo_al[variables_dispo_al].fillna(0)

# Les quantités sont sont en milliers de tonnes. Pour un affichage facile sur les g
# en divisant les quantités par 1000

dispo_al.iloc[:,  np.r_[3:5, 9:18]]  = dispo_al.iloc[:,  np.r_[3:5, 9:18]] / 1_000


# Renommer les variables
dispo_al.rename(columns={"Zone":"zone", "Produit": "produit", "Origine":"origine",
                         "Autres Utilisations":"autres_ut", "Disponibilité alimenta
                         "Disponibilité alimentaire en quantité (kg/personne/an)":"
                         "Disponibilité de matière grasse en quantité (g/personne/j
                         "Disponibilité de protéines en quantité (g/personne/jour)"
                         "Disponibilité intérieure": "dispo_int", "Exportations - Q
                         "Importations - Quantité": "importations", "Nourriture":"n
                         "Semences":"semences", "Traitement": "traitement",     "Va



dispo_al.loc[dispo_al["zone"] == "République populaire démocratique de Corée", "zon

# sauvegarder la table au format csv
dispo_al.to_csv("df_disponibilite_alimentaire.csv", index=False)


# Affichage les 5 premières lignes de la table
dispo_al.head()
```

Out[40]:

	zone	produit	origine	aliments_animaux	autres_ut	dispo_alim_kcal	dispo_ali
0	Afghanistan	Abats Comestible	animale	0.0	0.0	5.0	
1	Afghanistan	Agrumes, Autres	vegetale	0.0	0.0	1.0	
2	Afghanistan	Aliments pour enfants	vegetale	0.0	0.0	1.0	
3	Afghanistan	Ananas	vegetale	0.0	0.0	0.0	
4	Afghanistan	Bananes	vegetale	0.0	0.0	4.0	

1.4 - Analyse exploratoire du fichier sous nutrition

a) Chargement et description des tables

In [42]:
```
# Importation du fichier sous_nutrition.csv
sous_nutrit = pd.read_csv("sous_nutrition.csv")

# Afficher les dimensions du dataset
sous_nutrit.shape
```

Out[42]:
```
(1218, 3)
```

In [44]:
```
# Caractéristique de la table sous nutrition
sous_nutrit.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1218 entries, 0 to 1217
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Zone    1218 non-null   object
 1   Année   1218 non-null   object
 2   Valeur  624 non-null    object
dtypes: object(3)
memory usage: 28.7+ KB
```

In [46]:
```
# Afficher les dimensions de la table sur l'insécurité alimentaire
print("La table sur l'insécurité alimentaire comporte {} observations".format(sous_
print(" et {} variables".format(sous_nutrit.shape[1]))

# Caractéristiques des variables de la table sur l'insécurité alimentaire
print("La variable zone comporte", sous_nutrit.groupby("Zone")["Zone"].count().shap
print(sous_nutrit.groupby("Année")["Année"].count().shape[0], "modalités pour l'ann
```

La table sur l'insécurité alimentaire comporte 1218 observations et 3 variables
La variable zone comporte 203 modalités et 6 modalités pour l'année. Chaque pays est observé sur 6 années.

In [48]:
```
# Liste des zones dont on ne dispose aucune informations sur la sous nutrition

# Liste des zones de la table sous nutrition
df_sous_nutr = sous_nutrit.groupby("Zone")["Année"].count()
df_sous_nutr2 = df_sous_nutr.reset_index()
liste_zone_sous_nutr = df_sous_nutr2["Zone"].unique().tolist()


# liste des zones dont on ne dispose aucune informations sur la disponibilité alime
liste_zone_non_sous_nutr = []

for z in liste_zone_pop:
    if z not in df_sous_nutr:
        liste_zone_non_sous_nutr.append(z)

# Définition de la proportion de la population mondiale sur laquelle on ne dispose

pop_non_sous_nutr = pop.loc[pop["zone"].isin(liste_zone_non_sous_nutr), :]
pop_non_sous_nutr = pop_non_sous_nutr.groupby("année")["population"].sum()
pop_non_sous_nutr = pop_non_sous_nutr.reset_index()
pop_non_sous_nutr.rename(columns = {"population":"pop-sous_nutr_non_dispo"}, inplac

pop_mond_dispo_al["prop_pop_non_sous_nutr"] =  round(pop_non_sous_nutr["pop-sous_nu
```

In [50]:
```
print("Liste des zones  n'ayant aucune information sur l'insécurité alimentaire ",
print("soit un total de : ", len(liste_zone_non_sous_nutr), "zones sur 236 représen
```

```
Liste des zones  n'ayant aucune information sur l'insécurité alimentaire  ['Anguill
a', 'Antilles néerlandaises (ex)', 'Aruba', 'Bonaire, Saint-Eustache et Saba', 'Cura
çao', 'Gibraltar', 'Guadeloupe', 'Guam', 'Guyane française', 'Liechtenstein', 'Marti
nique', 'Mayotte', 'Monaco', 'Montserrat', 'Réunion', 'Sahara occidental', 'Saint-Ba
rthélemy', 'Saint-Marin', 'Saint-Martin (partie française)', 'Saint-Pierre-et-Miquel
on', 'Saint-Siège', 'Sainte-Hélène, Ascension et Tristan da Cunha', 'Sint Maarten
(partie néerlandaise)', 'Île de Man', 'Îles Anglo-Normandes', 'Îles Caïmanes', 'Îles
Falkland (Malvinas)', 'Îles Féroé', 'Îles Mariannes du Nord', 'Îles Turques-et-Caïqu
es', 'Îles Vierges américaines', 'Îles Vierges britanniques', 'Îles Wallis-et-Futun
a']
soit un total de :  33 zones sur 236 représentant 0,06% de la population mondiale en
2017
```

In [52]:
```
pop_mond_dispo_al.head()
```

Out[52]:

	année	pop-dispo_al	pop_mond_totale	prop_pop_disp_al	prop_pop_non_disp_al	prop_p
0	2013	6.944803e+09	7.210846e+09	96.31	3.69	
1	2014	7.025939e+09	7.295558e+09	96.30	3.70	
2	2015	7.106663e+09	7.380067e+09	96.30	3.70	
3	2016	7.186864e+09	7.464295e+09	96.28	3.72	
4	2017	7.266471e+09	7.548134e+09	96.27	3.73	

In [54]:

```
# Afficher les 5 premières lignes de la table

sous_nutrit.head()
```

Out[54]:

	Zone	Année	Valeur
0	Afghanistan	2012-2014	8.6
1	Afghanistan	2013-2015	8.8
2	Afghanistan	2014-2016	8.9
3	Afghanistan	2015-2017	9.7
4	Afghanistan	2016-2018	10.5

b) Standardisation et renommage des variables

In [56]:

```
# Conversion de la colonne valeur (avec l'argument errors=coerce qui permet de conv
sous_nutrit["Valeur"] = pd.to_numeric(sous_nutrit["Valeur"], errors="coerce")


# Puis remplacement des NaN en 0
sous_nutrit["Valeur"] = sous_nutrit["Valeur"].fillna(0)


# Renommage des variables
sous_nutrit.rename(columns={"Zone":"zone", "Année":"periode", "Valeur":"pop_sous_al



# Transformation des valeurs de la variables sur la population en unités
sous_nutrit["pop_sous_alim"] = sous_nutrit["pop_sous_alim"] * 1_000_000


# Définition d'une liste des périodes
liste_periode = sous_nutrit["periode"].unique().tolist()


# Création de la variable année qui indique sur l'années de chaque période prise en
sous_nutrit["année"] = "2013"
```

```
a = 1
for pde in liste_periode:
    sous_nutrit.loc[sous_nutrit["periode"] == pde, "année"] = 2012 + a
    a += 1

# Exclusion des informations sur l'année 2018 car l'analyse porte sur la période 20
sous_nutrit = sous_nutrit[["zone", "periode", "année", "pop_sous_alim"]]
sous_nutrit = sous_nutrit.loc[sous_nutrit["année"] != 2018, :]



sous_nutrit.loc[sous_nutrit["zone"] == "République populaire démocratique de Corée"

# Sauvegarder la table au format csv
sous_nutrit.to_csv("df_sous_nutrition.csv", index=False)


# Actualisation du dictionnaire
dict_variables["pop_sous_alim"] = "Nombre de personne en état de sous nutrition"


# Afficher les 5 premières lignes de la table sous nutrition
sous_nutrit.head()
```

Out[56]:

	zone	periode	année	pop_sous_alim
0	Afghanistan	2012-2014	2013	8600000.0
1	Afghanistan	2013-2015	2014	8800000.0
2	Afghanistan	2014-2016	2015	8900000.0
3	Afghanistan	2015-2017	2016	9700000.0
4	Afghanistan	2016-2018	2017	10500000.0

1.5 - Analyse exploratoire du fichier aide alimentaire

a) Chargement, description et renommage des variables

In [58]:

```
# Importation  de la table sur l'aide alimentaire
aide_al = pd.read_csv("aide_alimentaire.csv")

#Afficher les dimensions de la table
aide_al.shape
```

Out[58]: (1475, 4)

In [60]:

```
# Caractéristique de la table aide alimentaire
```

```
aide_al.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1475 entries, 0 to 1474
Data columns (total 4 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   Pays bénéficiaire  1475 non-null   object
 1   Année              1475 non-null   int64
 2   Produit            1475 non-null   object
 3   Valeur             1475 non-null   int64
dtypes: int64(2), object(2)
memory usage: 46.2+ KB
```

In [62]:

```
# Afficher les dimensions de la table sur l'aide alimentaire
print("La table sur l'aide alimentaire comporte {} observations".format(aide_al.sha
print(" et {} variables".format(aide_al.shape[1]))

# Caractéristiques des variables de la table disponibilité alimentaire
print("Les variables Année et valeur sont de type", aide_al["Année"].dtype, end="")
print(". Les variables Pays bénéficiaire et Produit sont de type", aide_al["Produit
print(". Avec", aide_al.groupby("Pays bénéficiaire")["Pays bénéficiaire"].count().s
print(" et", aide_al.groupby("Produit")["Produit"].count().shape[0], "pour le Produ
```

```
La table sur l'aide alimentaire comporte 1475 observations et 4 variables
Les variables Année et valeur sont de type int64. Les variables Pays bénéficiaire et
Produit sont de type object. Avec 76 modalités pour la variable Pays benficiaire et
16 pour le Produit
```

In [64]:

```
# Renommer les variables
aide_al.rename(columns={"Pays bénéficiaire":"zone", "Année":"année", "Produit":"pro


# Transformation des quantités d'aide alimentaire des tonnes en KG
#aide_al["quantité"]  = aide_al["quantité"] * 1_000


# Exclusion des informations sur 2018
aide_al = aide_al.loc[aide_al["année"] != 2018, :]


aide_al.loc[aide_al["zone"] == "République populaire démocratique de Corée", "zone"

# Sauvegarder la table au format csv
aide_al.to_csv("df_aide_alimentaire.csv", index=False)


# Affichage les 5 premières lignes de la table
aide_al.head()
```

Out[64]:

	zone	année	produit	quantité
0	Afghanistan	2013	Autres non-céréales	682
1	Afghanistan	2014	Autres non-céréales	335
2	Afghanistan	2013	Blé et Farin	39224
3	Afghanistan	2014	Blé et Farin	15160
4	Afghanistan	2013	Céréales	40504

II - Construction des indicateurs et analyse approfondie des données

2.1 - Définition des fonctions pour afficher les graphiques

In [66]:

```
# Définition de fonction pour afficher les diagrammes à barres

def plot_countries_by_level_indicator(data_frame: pd.DataFrame,
                                        column_name: str = "nourriture",
                                        n_countries: int = 10,
                                        y_axis_limits: int = 100,
                                        order: str = "head",
                                        ylabel: str = "show a label",
                                        title: str = "bar",
                                        fig_dimension: tuple = (12, 6)) -> None:
    """
    Plot a bar chart of the top countries by food allocation percentage.

    Args:
        data_frame (pd.DataFrame): DataFrame containing the food allocation data
        column_name (str): Column name for food allocation values
        n_countries (int): Number of top countries to display

    Returns:
        None: Displays the plot
    """

    # Get top countries by food allocation
    if order == "head":
        order_data = data_frame.sort_values(column_name, ascending=False).head(n_co
    elif order == "tail":
        order_data = data_frame.sort_values(column_name, ascending=False).tail(n_co

    # Create the plot
    plt.figure(figsize=fig_dimension)
    plt.bar(height=order_data[column_name], x=order_data["zone"])

    # Set y-axis limits
```

```
    plt.ylim(0, y_axis_limits)

    # Add title and labels
    plt.title(title, color="green", fontweight="bold", fontname="Arial", fontsize=1
    plt.xlabel("Pays", fontweight="bold")
    plt.ylabel(ylabel, fontweight="bold")

    # Rotate x-axis labels for better readability
    plt.xticks(rotation=45, ha='right')

    # Add value labels on top of each bar
    values = order_data[column_name].tolist()
    for i, value in enumerate(values):
        plt.text(i-0.3, value+1, round(value))

    plt.show()
```

In [97]:

```
# Définition d'une fonction pour catégoriser le niveau de prévalence de sous nutrit

def categorize_malnutrition(df: pd.DataFrame) -> pd.DataFrame:
    """
    Categorizes zones based on their malnutrition rates.

    Args:
        df (pd.DataFrame): DataFrame containing malnutrition data with 'zone' and '

    Returns:
        pd.DataFrame: DataFrame with added categorization columns 'prev_sous_nutr'
    """
    # Define categorization thresholds and labels
    categories = [
        (0, 0, "Pas de sous nutrition", "Pas de sous nutrition"),
        (0, 5, "Moins de 5%", "Tres faible"),
        (5, 10, "5 - 10%", "Faible"),
        (10, 20, "10 - 20%", "Modéré"),
        (20, 30, "20 - 30%", "Elevé"),
        (30, float('inf'), "Plus de 30%", "Tres élevé")
    ]

    # Create a copy to avoid modifying the original dataframe
    result_df = df.copy()

    # Apply categorization for each unique zone
    for zone in result_df["zone"].unique():
        # Get the malnutrition rate for this zone
        mal_rate = result_df.loc[result_df["zone"] == zone, "prop_pop_sous_al"].val

        # Find the appropriate category
        for lower, upper, label1, label2 in categories:
            if lower <= mal_rate <= upper:
                result_df.loc[result_df["zone"] == zone, "prev_sous_nutr"] = label1
                result_df.loc[result_df["zone"] == zone, "prev_sous_nutr_2"] = labe
                break
```

```
    return result_df
```

In [99]:
```
# Définition d'une fonction qui affiche la proportion du type de produit

def plot_pie_chart(
    data: pd.DataFrame,
    category_column: str,
    title: str = "Pie Chart",
    figsize: tuple = (13, 6),
    colors: list = None,
    **kwargs
) -> None:
    """
    Creates a pie chart showing the proportions of categories in a DataFrame column

    Args:
        data (pd.DataFrame): DataFrame containing the data to plot
        category_column (str): Name of the column to use for categories
        title (str, optional): Title for the pie chart. Defaults to "Pie Chart".
        figsize (tuple, optional): Figure size as (width, height). Defaults to (13,
        colors (list, optional): List of colors for pie slices. Defaults to None.
        **kwargs: Additional keyword arguments to pass to plt.pie()

    Returns:
        None: This function displays a plot but doesn't return any value.

    Raises:
        KeyError: If the specified category_column doesn't exist in the DataFrame
    """
    if category_column not in data.columns:
        raise KeyError(f"Column '{category_column}' not found in DataFrame")

    # Count occurrences of each category
    value_counts = data[category_column].value_counts()

    plt.figure(figsize=figsize)
    plt.pie(
        x=value_counts.values,
        labels=value_counts.index,
        autopct='%.1f%%',
        colors=colors,
        **kwargs
    )
    plt.title(title, color="green", fontweight="bold", fontname="Arial", fontsize=1
    plt.axis('equal')  # Equal aspect ratio ensures that pie is drawn as a circle
```

2.2 - Calcul des indicateurs de sous alimentation

a) Construction de la table appropriée

In [101…

```
# Chargement des tables
df_pop = pd.read_csv("df_population.csv")
df_sous_nutrit = pd.read_csv("df_sous_nutrition.csv")


# Jointure des tables population et sous nutrtion
df_sous_nutrition = pd.merge(df_sous_nutrit, df_pop, on = ["zone", "année"], how="l


# description de la table
df_sous_nutrition.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1015 entries, 0 to 1014
Data columns (total 5 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   zone           1015 non-null   object
 1   periode        1015 non-null   object
 2   année          1015 non-null   int64
 3   pop_sous_alim  1015 non-null   float64
 4   population     1015 non-null   float64
dtypes: float64(2), int64(1), object(2)
memory usage: 39.8+ KB
```

Pour vérifier si les données sont completes, on va identifier les pays qui ont moins de 5 années d'observation, car chaque pays est observé sur 5 ans. De 2013 à 2017

In [104…

```
# Nombre de pays ayant moins de 5 années d'observation.
verif_tab = df_sous_nutrition.groupby("zone")["année"].count().reset_index()

print("Le nombre de pays ayant moins de 5 ans d'observation est de", verif_tab.loc[
```

```
Le nombre de pays ayant moins de 5 ans d'observation est de 0 . Les données sont don
c complètes
```

In [106…

```
# Affichage du nouveau dataset
df_sous_nutrition.head()
```

Out[106…

	zone	periode	année	pop_sous_alim	population
0	Afghanistan	2012-2014	2013	8600000.0	32269589.0
1	Afghanistan	2013-2015	2014	8800000.0	33370794.0
2	Afghanistan	2014-2016	2015	8900000.0	34413603.0
3	Afghanistan	2015-2017	2016	9700000.0	35383032.0
4	Afghanistan	2016-2018	2017	10500000.0	36296113.0

b) Construction des indicateurs

i) Nombre de personnes en sous alimentation

In [110…

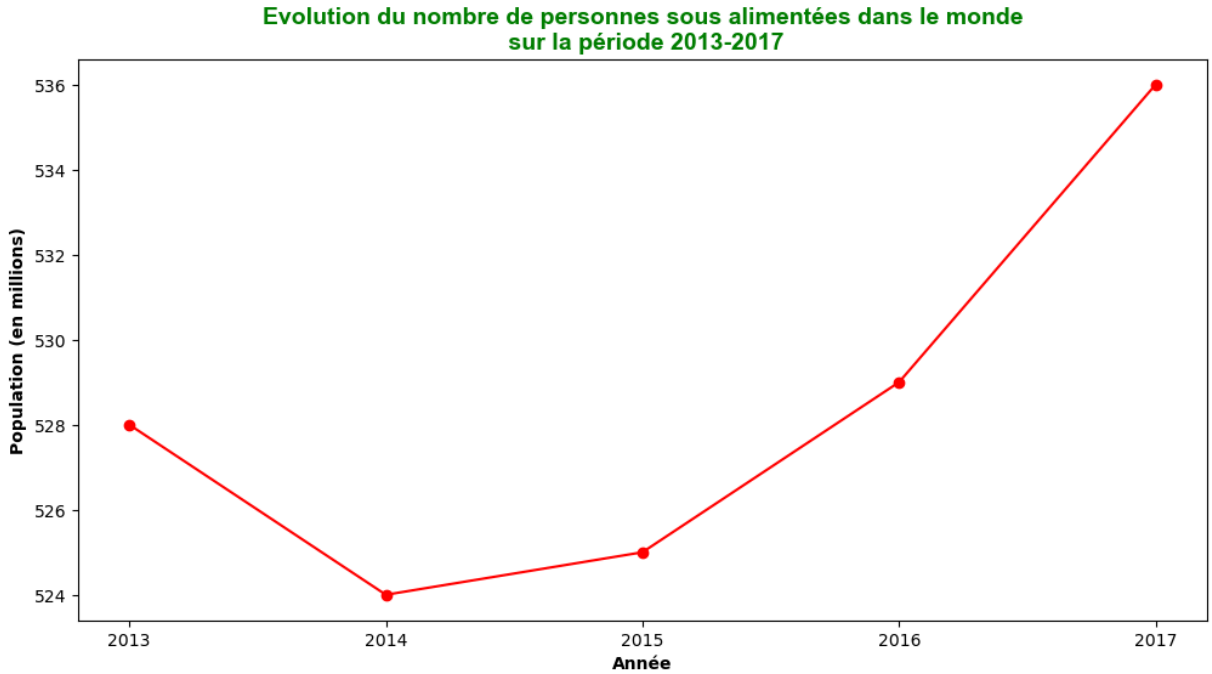
```
# Nombre de personnes en sous alimentation dans le monde sur la période 2013-2017
sous_al_ann = df_sous_nutrition.groupby("année")["pop_sous_alim"].sum().reset_index
sous_al_ann.sort_values("pop_sous_alim", ascending=False)


# Mettre la variable sur le nombre de personne sous alimentée en millions
sous_al_ann["pop_sous_alim_2"] = round(sous_al_ann["pop_sous_alim"] / 1_000_000)
```

In [112…

```
# Affichage de l'évolution du nombre de personne en sous alimentation dans le monde
plt.figure(figsize=(12,6))
plt.plot(sous_al_ann["année"], sous_al_ann["pop_sous_alim_2"], marker='o', color='r
plt.title("Evolution du nombre de personnes sous alimentées dans le monde\n sur la
          color="green", fontweight="bold", fontname="Arial", fontsize=14)
plt.xlabel("Année", fontweight ="bold")
plt.ylabel("Population (en millions)", fontweight ="bold")
plt.xticks([2013, 2014, 2015, 2016, 2017])
```

Out[112…

```
([<matplotlib.axis.XTick at 0x1c22f552600>,
  <matplotlib.axis.XTick at 0x1c22f39a4b0>,
  <matplotlib.axis.XTick at 0x1c22f1fd0d0>,
  <matplotlib.axis.XTick at 0x1c22f55a630>,
  <matplotlib.axis.XTick at 0x1c22f55af60>],
 [Text(2013, 0, '2013'),
  Text(2014, 0, '2014'),
  Text(2015, 0, '2015'),
  Text(2016, 0, '2016'),
  Text(2017, 0, '2017')])
```



ii) Proportion de la population mondiale en état de sous alimentation sur la

période 2013-2017

In [115…

```
# Calcul de la proportion de personnes en sous alimentation dans chaque pays chaque
df_sous_nutrition["prop_pop_sous_al"] = round((df_sous_nutrition["pop_sous_alim"] /


# Sauvegarder la table au format csv
df_sous_nutrition.to_csv("df_sous_nutrition_2.csv", index=False)
```

In [117…

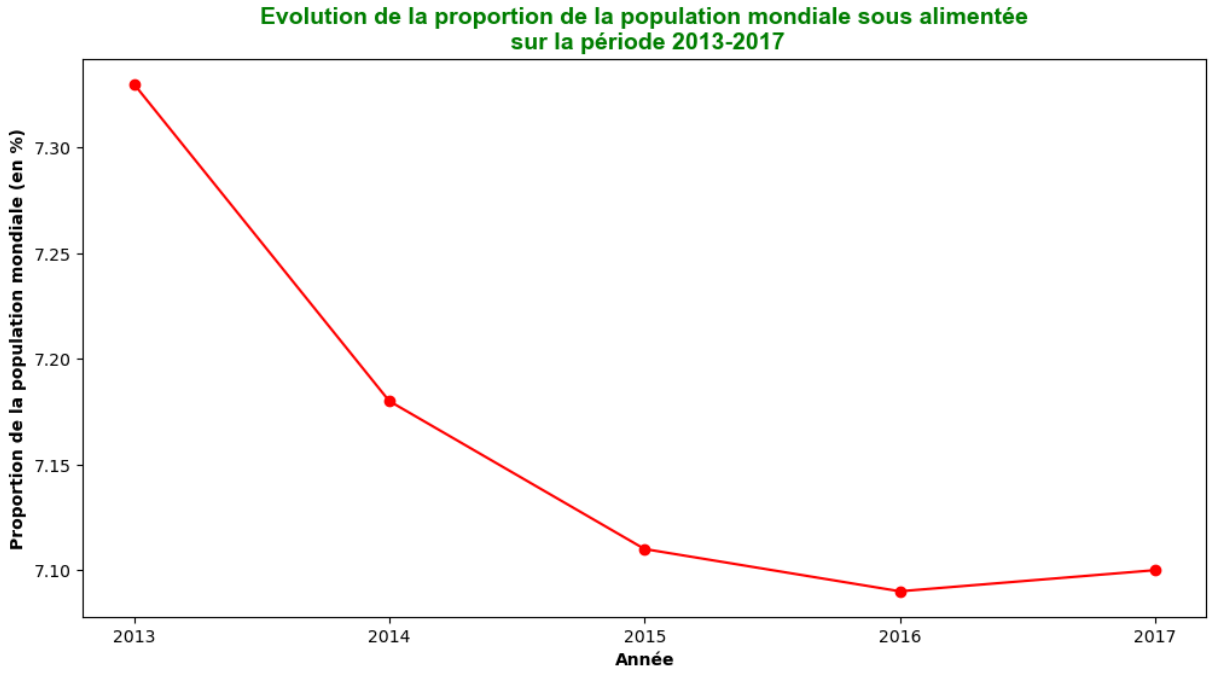
```
# Calcul de la proportion de la population mondiale sous alimentée

# méthode 1
sous_al_prop_mon_1 = df_sous_nutrition.groupby("année")["prop_pop_sous_al"].mean().


# méthode 2
# calcul de la population mondiale totale et et la population mondiale sous aliment
pop_mon_sous_al = df_sous_nutrition.groupby("année")["pop_sous_alim"].sum().reset_i
pop_mon_tot = df_sous_nutrition.groupby("année")["population"].sum().reset_index()

# jointure des deux populations
sous_al_prop_mon_2 = pd.merge(pop_mon_tot, pop_mon_sous_al, on ="année")

# Calcul de la proportion de la population
sous_al_prop_mon_2["prop_pop_sous_al"] = round((sous_al_prop_mon_2["pop_sous_alim"]
```

In [119…

```
# Evolution de la proportion de la population mondiale en sous alimentation sur la
plt.figure(figsize=(12,6))
plt.plot(sous_al_prop_mon_2["année"], sous_al_prop_mon_2["prop_pop_sous_al"], marke
plt.title("Evolution de la proportion de la population mondiale sous alimentée\n su
          color="green", fontweight="bold", fontname="Arial", fontsize=14)
plt.xlabel("Année", fontweight ="bold")
plt.ylabel("Proportion de la population mondiale (en %)", fontweight ="bold")
plt.xticks([2013, 2014, 2015, 2016, 2017])
```

Out[119…

```
([<matplotlib.axis.XTick at 0x1c22f565490>,
  <matplotlib.axis.XTick at 0x1c22f566240>,
  <matplotlib.axis.XTick at 0x1c22f5656a0>,
  <matplotlib.axis.XTick at 0x1c22f5663f0>,
  <matplotlib.axis.XTick at 0x1c231058a40>],
 [Text(2013, 0, '2013'),
  Text(2014, 0, '2014'),
  Text(2015, 0, '2015'),
  Text(2016, 0, '2016'),
  Text(2017, 0, '2017')])
```

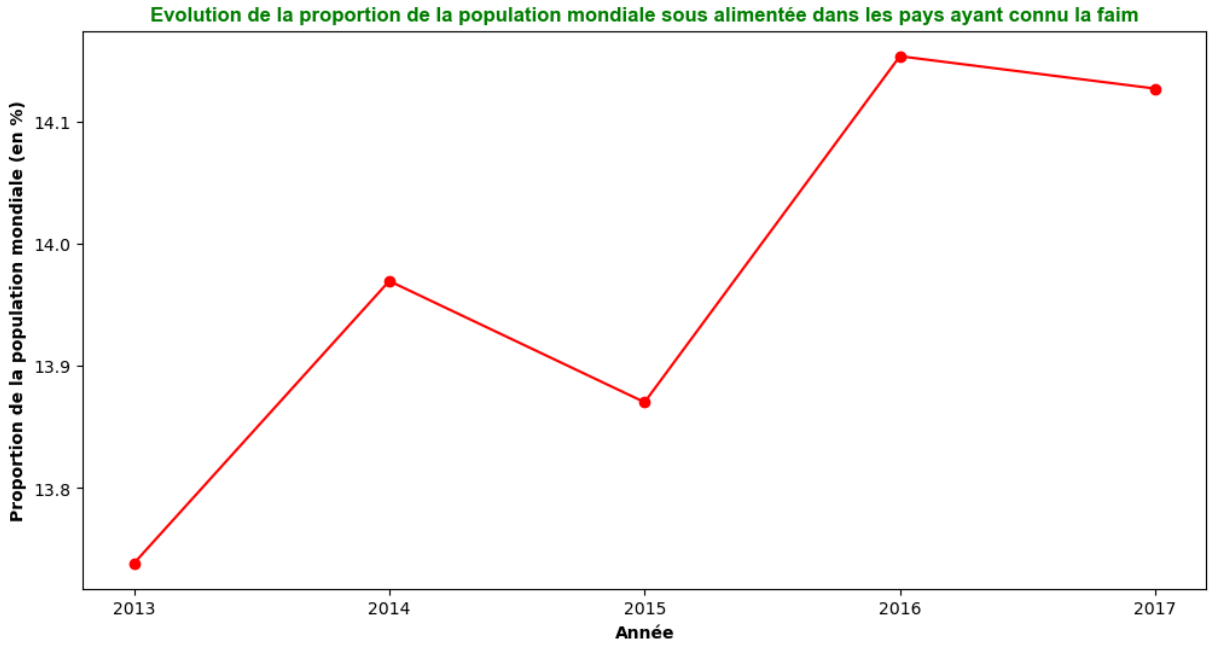


In [121…

```
# Proportion de la population mondiale sous alimentée chaque année en excluant les

# Exclusion des zones du monde qui n'ont jamais connu la faim sur la période 2013-2
df_sous_nutrt = df_sous_nutrition.loc[df_sous_nutrition["prop_pop_sous_al"] != 0, :

sous_al_prop_mon = df_sous_nutrt.groupby("année")["prop_pop_sous_al"].mean().reset_

sous_al_prop_mon
# Evolution de la proportion de la population mondiale en sous alimentation sur la

plt.figure(figsize=(12,6))
plt.plot(sous_al_prop_mon["année"], sous_al_prop_mon["prop_pop_sous_al"], marker='o
plt.title("Evolution de la proportion de la population mondiale sous alimentée dans
          color="green", fontweight="bold", fontname="Arial", fontsize=12)
plt.xlabel("Année", fontweight ="bold")
plt.ylabel("Proportion de la population mondiale (en %)", fontweight ="bold")
plt.xticks([2013, 2014, 2015, 2016, 2017])
```

Out[121…

```
([<matplotlib.axis.XTick at 0x1c22f55d9a0>,
  <matplotlib.axis.XTick at 0x1c22f55da60>,
  <matplotlib.axis.XTick at 0x1c22f566630>,
  <matplotlib.axis.XTick at 0x1c23106e3f0>,
  <matplotlib.axis.XTick at 0x1c23107e9f0>],
 [Text(2013, 0, '2013'),
  Text(2014, 0, '2014'),
  Text(2015, 0, '2015'),
  Text(2016, 0, '2016'),
  Text(2017, 0, '2017')])
```



In [123…

```
# calcul du niveau de prévalence de sous nutrition en 2017 dans chaque zone
sous_al_pays_17 = df_sous_nutrition.loc[df_sous_nutrition["année"] == 2017, : ]

# Calcul du niveau de prévalence de sous nutrition moyen sur la période 2013-2017 d
sous_al_pays = df_sous_nutrition.groupby("zone")["prop_pop_sous_al"].mean().reset_i
```

In [125…

```
sous_al_pays.head()
```

Out[125…

	zone	prop_pop_sous_al
0	Afghanistan	27.044
1	Afrique du Sud	4.904
2	Albanie	3.456
3	Algérie	3.274
4	Allemagne	0.000

In [127…

```
# Nombre de pays en situation de sous nutrition, en fonction du niveau de prévalenc

# La fonction categorize_malnutrition catégorise les niveaux de prevalence de sous
sous_al_pays_2 = categorize_malnutrition(sous_al_pays)


# Comptage du nombre de pays dans chaque niveau de prévalence
nb_pays_sous_nutrt = sous_al_pays_2.groupby(["prev_sous_nutr", "prev_sous_nutr_2"])



# Ordonner les modaliés du niveau de sous nutrition
ordre_niv_sous_nutr = ["Pas de sous nutrition", "Moins de 5%", "5 - 10%", "10 - 20%
nb_pays_sous_nutrt["prev_sous_nutr"] = pd.Categorical(nb_pays_sous_nutrt["prev_sous
nb_pays_sous_nutrt = nb_pays_sous_nutrt.sort_values("prev_sous_nutr")
```

```
# Affichage du dataset
nb_pays_sous_nutrt.head()
```

Out[127…

	prev_sous_nutr	prev_sous_nutr_2	nombre_de_pays
4	Pas de sous nutrition	Pas de sous nutrition	117
3	Moins de 5%	Tres faible	20
2	5 - 10%	Faible	22
0	10 - 20%	Modéré	25
1	20 - 30%	Elevé	11

In [129…

```
# Nombre de pays en situation de sous alimentation en 2017, selon  niveau de préval

sous_al_pays_17_2 = categorize_malnutrition(sous_al_pays_17)
nb_pays_sous_nutrt_17 = sous_al_pays_17_2.groupby(["prev_sous_nutr", "prev_sous_nut



# Ordonner les modaliés du niveau de sous nutrition
nb_pays_sous_nutrt_17["prev_sous_nutr"] = pd.Categorical(nb_pays_sous_nutrt_17["pre
nb_pays_sous_nutrt_17 = nb_pays_sous_nutrt_17.sort_values("prev_sous_nutr")


nb_pays_sous_nutrt_17.head()
```

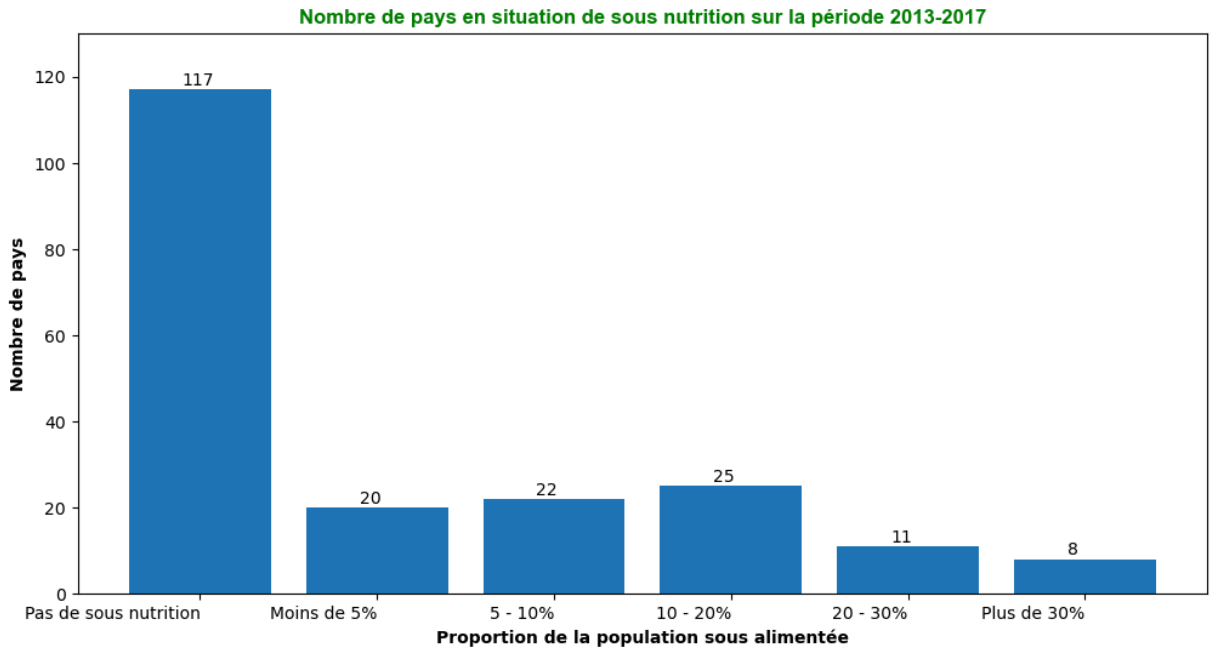
Out[129…

	prev_sous_nutr	prev_sous_nutr_2	nombre_de_pays
4	Pas de sous nutrition	Pas de sous nutrition	119
3	Moins de 5%	Tres faible	17
2	5 - 10%	Faible	25
0	10 - 20%	Modéré	23
1	20 - 30%	Elevé	10

In [131…

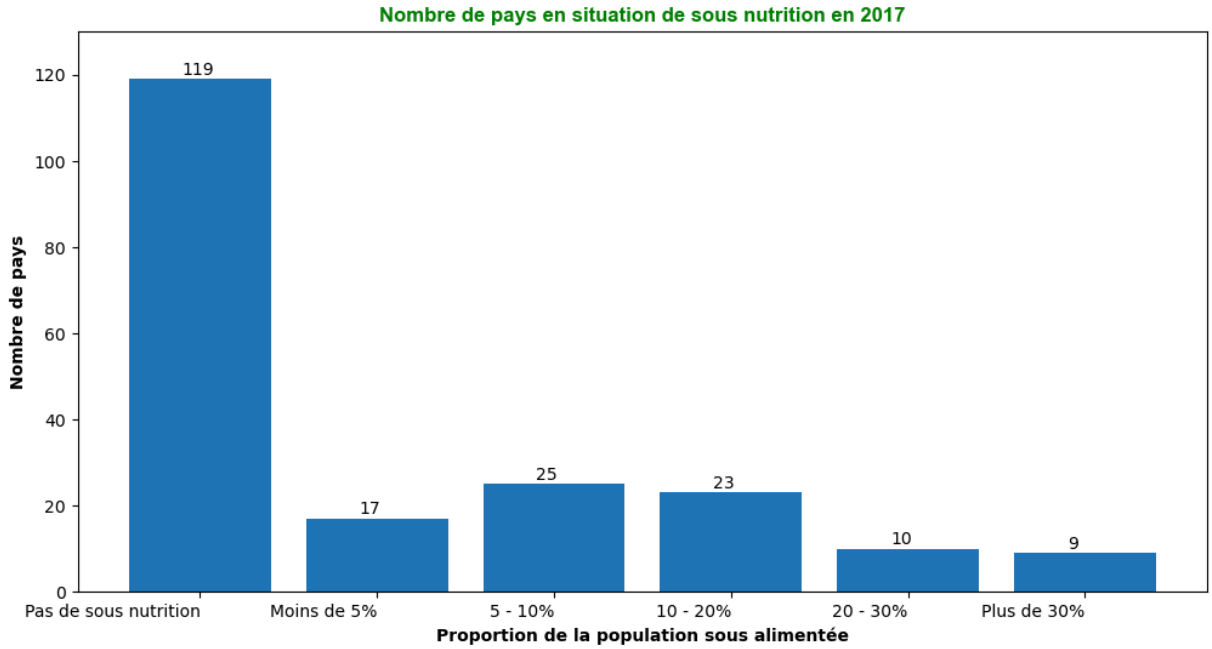
```
# Affichage du nombre de pays en situation de sous alimentation sur la période 2013

plt.figure(figsize=(12,6))
plt.bar(height=nb_pays_sous_nutrt["nombre_de_pays"], x=nb_pays_sous_nutrt["prev_sou
plt.ylim(0, 130) # Pour préciser l'étendu des valeurs de l'axes des ordonnées

plt.title("Nombre de pays en situation de sous nutrition sur la période 2013-2017",
          color="green", fontweight="bold", fontname="Arial", fontsize=12)
plt.xlabel("Proportion de la population sous alimentée", fontweight ="bold")
plt.ylabel("Nombre de pays", fontweight ="bold")
plt.xticks(rotation=0, ha='right')  # ha='right' = alignement à droite


# Afficher les valeurs associées à chaque niveau
```

```
val_niv = nb_pays_sous_nutrt["nombre_de_pays"].tolist()
for i in range(6):
    plt.text(i-0.1, val_niv[i]+1, round(val_niv[i]))
plt.show()
```



In [133…

```
# Affichage du nombre de pays en situation de sous alimentation en 2017, en fonctio

plt.figure(figsize=(12,6))
plt.bar(height=nb_pays_sous_nutrt_17["nombre_de_pays"], x=nb_pays_sous_nutrt_17["pr
plt.ylim(0, 130) # Pour préciser l'étendu des valeurs de l'axes des ordonnées

plt.title("Nombre de pays en situation de sous nutrition en 2017",
          color="green", fontweight="bold", fontname="Arial", fontsize=12)
plt.xlabel("Proportion de la population sous alimentée", fontweight ="bold")
plt.ylabel("Nombre de pays", fontweight ="bold")
plt.xticks(rotation=0, ha='right')  # ha='right' = alignement à droite


# Afficher les valeurs associées à chaque niveau

val_niv = nb_pays_sous_nutrt_17["nombre_de_pays"].tolist()
for i in range(6):
    plt.text(i-0.1, val_niv[i]+1, round(val_niv[i]))
plt.show()
```

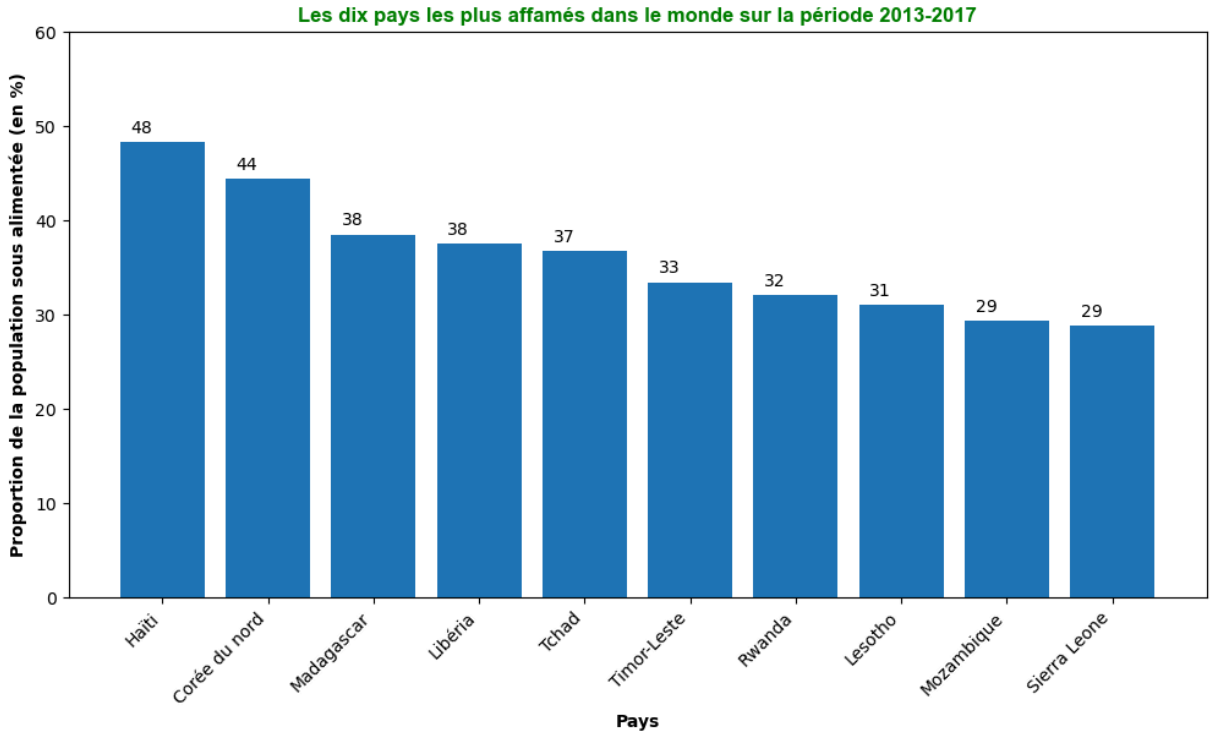


In [135…

```
# Top 10 des pays les plus affamés dans le monde sur la période 2013-2017, selon la

sous_al_prop_mon_3 = df_sous_nutrt.groupby("zone")["prop_pop_sous_al"].mean().reset

top_pays_sous_al = sous_al_prop_mon_3.sort_values("prop_pop_sous_al", ascending=Fal
```

In [137…

```
# Affichage des 10 pays les plus affamés

plot_countries_by_level_indicator(data_frame=top_pays_sous_al,
                                  column_name="prop_pop_sous_al",
                                  n_countries=10,
                                  y_axis_limits=60,
                                  order="head",
                                  ylabel="Proportion de la population sous
                                  title="Les dix pays les plus affamés dans
```

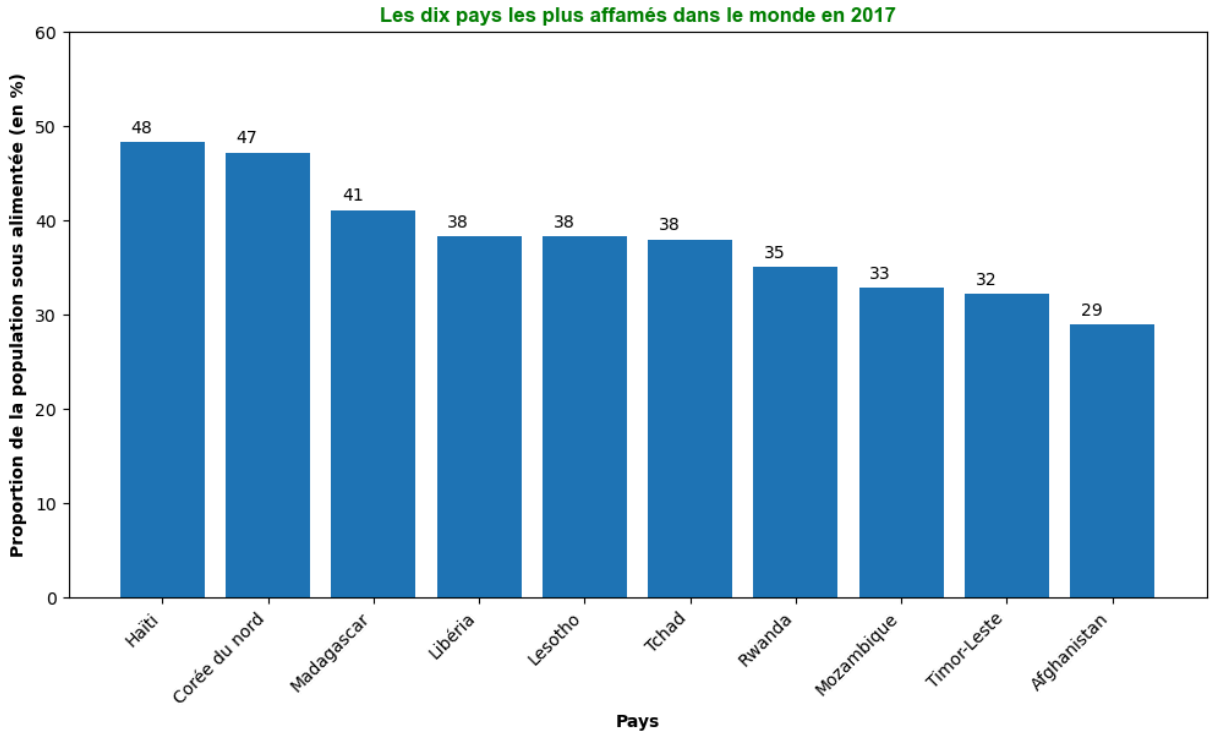


In [139…

```
# Top 10 des pays les plus affamés dans le monde  en 2017, selon la proportion de l

sous_al_prop_mon_3_17 = df_sous_nutrt.loc[df_sous_nutrt["année"] == 2017, :]

top_pays_sous_al_17 = sous_al_prop_mon_3_17.sort_values("prop_pop_sous_al", ascendi
```

In [141…

```
# Affichage des résultats

plot_countries_by_level_indicator(data_frame=top_pays_sous_al_17,
                                  column_name="prop_pop_sous_al",
                                  n_countries=10,
                                  y_axis_limits=60,
                                  order="head",
                                  ylabel="Proportion de la population sous
                                  title="Les dix pays les plus affamés dans
```



2.3 - Calcul de la capacité alimentaire

In [144…

```
# Chargement de la table disponibilité alimentaire
df_dispo_al = pd.read_csv("df_disponibilite_alimentaire.csv")

# Calcul de la disponibilité alimentaire totale en kcal par jour  et  en fonction d
dispo_alim_zone = df_dispo_al[["zone", "origine", "dispo_alim_kcal"]].pivot_table(
    index="zone", columns="origine", values="dispo_alim_kcal", aggfunc="sum")


dispo_alim_zone = dispo_alim_zone.reset_index()
dispo_alim_zone.rename(columns={"animale":"dispo_al_animal", "vegetale":"dispo_al_v


# Calcul de la disponibilité alimentaire totale
dispo_alim_zone["dispo_al_totale"] = dispo_alim_zone["dispo_al_animal"] + dispo_ali


# Jointure avec la table population
df_pop = pd.read_csv("df_population.csv")
df_pop_17 = df_pop.loc[df_pop["année"] == 2017, ["zone", "population"]]
dispo_alim_zone_17 = pd.merge(dispo_alim_zone , df_pop_17, on ="zone", how="left")

# dict_variables

# Affichage de la table
dispo_alim_zone_17.head()
```

Out[144…

	zone	dispo_al_animal	dispo_al_vegetale	dispo_al_totale	population
0	Afghanistan	216.0	1871.0	2087.0	36296113.0
1	Afrique du Sud	487.0	2533.0	3020.0	57009756.0
2	Albanie	985.0	2203.0	3188.0	2884169.0
3	Algérie	378.0	2915.0	3293.0	41389189.0
4	Allemagne	1042.0	2461.0	3503.0	82658409.0

a) - Estimation du nombre de personnes pouvant etre nourries

Selon les organisations onusienne (FAO, PAM) en charge des questions alimetaires dans le monde, il faudra en moyenne 2100 kcal/jour/ à une personne pour son alimentation.

Le nombre de personne pouvant théoriquement etre nourrie est égale au rapport entre la disponibilité alimentaire totale pour l'ensemble des individus et le nombre de Kcal necessaire pour une bonne alimentation.

In [148…

```
dispo_alim_zone_17[["dispo_al_animal", "dispo_al_vegetale"]].sum()
```

Out[148…

```
dispo_al_animal       96660.0
dispo_al_vegetale    398782.0
dtype: float64
```

In [150…

```
# Calcul du nombre de personnes pouvant etre nourri dans chaque pays en 2017

dispo_alim_zone_17["nb_pers_nourrie"] = round(dispo_alim_zone_17["dispo_al_totale"]
```

In [152…

```
print("En 2017 la population mondiale était de ", dispo_alim_zone_17["population"].
print("Les disponibilités alimentaires cette année permettaient de nourrir au total
print("soit ", round(dispo_alim_zone_17["nb_pers_nourrie"].sum() / dispo_alim_zone_
```

```
En 2017 la population mondiale était de  7291900830.0 personnes.
Les disponibilités alimentaires cette année permettaient de nourrir au total 9961421
245.0 personnes.
soit  1.37 fois plus de personnes pouvant etre nourrie.
```

Les disponibilités alimentaires mondiales de 2017 permettait donc, non seulement de couvrir les besoins alimentaires mondiaux, mais aussi de nourrir en plus 37% de la population mondiale.

b) - Estimation du nombre de personnes pouvant etre nourries avec les produits végétaux

In [156…

```
# Calcul du nombre de personne pouvant etre nourrie en 2017 uniquement par des prod
dispo_alim_zone_17["nb_pers_nour_veg"] = round(dispo_alim_zone_17["dispo_al_vegetal
```

In [158…

```
print("Lorsqu'on considére uniquement les produits d'origine végétale, on pourrait
print("Soit au total", dispo_alim_zone_17["nb_pers_nour_veg"].sum(), "personnes.")
```

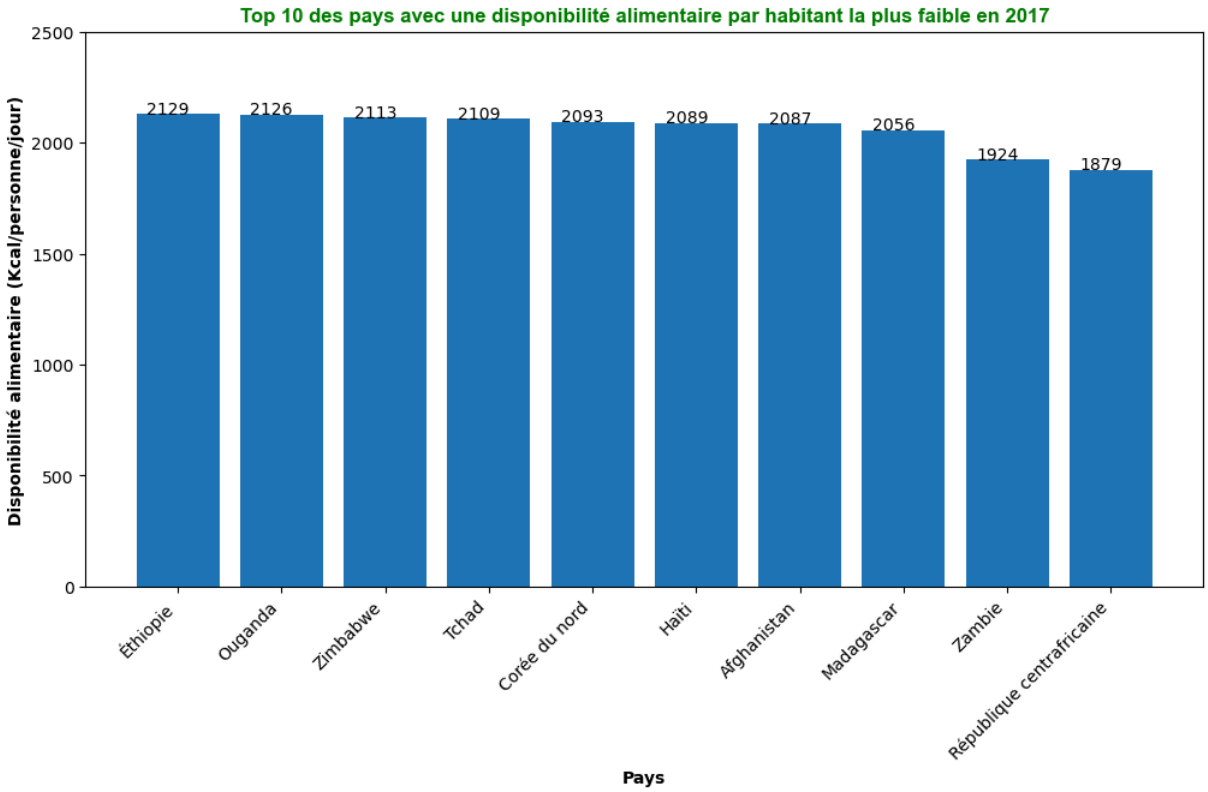
```
Lorsqu'on considére uniquement les produits d'origine végétale, on pourrait nourrir
1.13 fois plus de personnes.
Soit au total 8219411527.0 personnes.
```

Meme lorsqu'on considere uniquement les produits alimentaires, les disponibilitées alimentaires de 2017 permettaient de nourrir 13% de la population mondiale en plus.

c) - Disponibilité alimentaire par personne selon le pays

In [162…

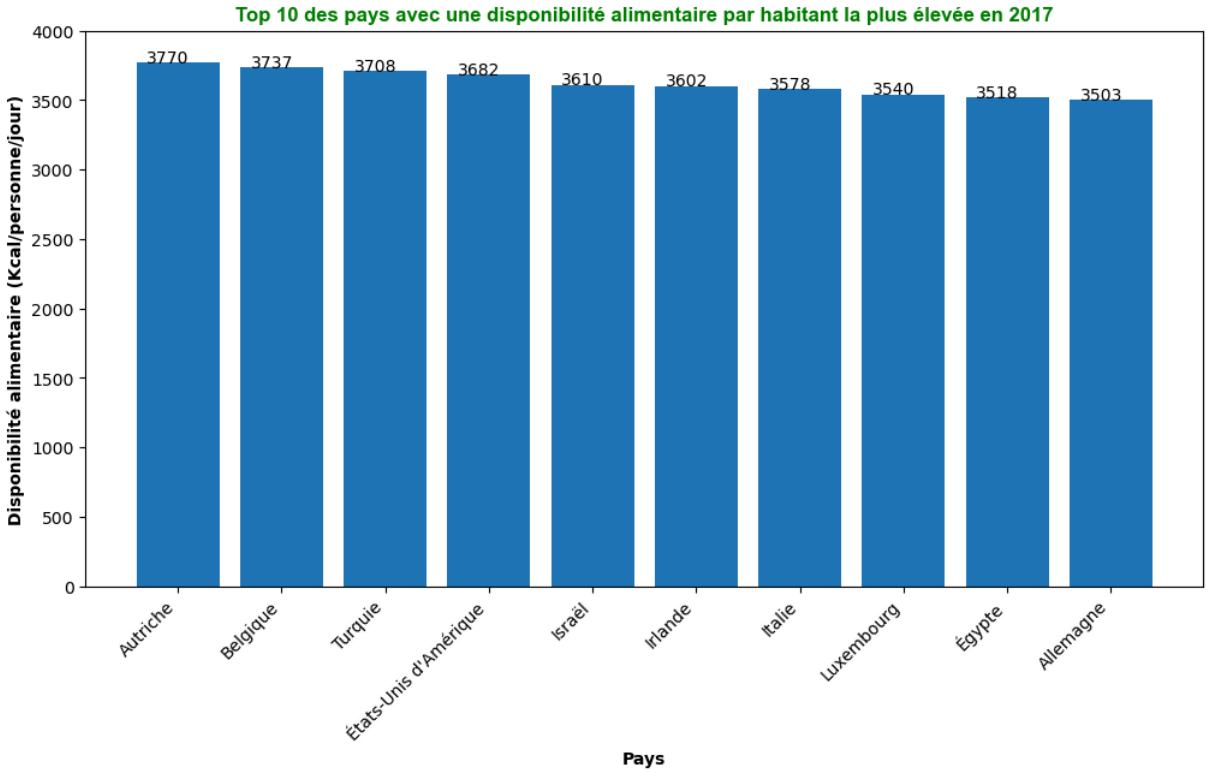
```
# Affichage des 10 pays qui ont le moins de disponibilité alimentaire par personne

plot_countries_by_level_indicator(data_frame=dispo_alim_zone,
                                  column_name="dispo_al_totale",
                                  n_countries=10,
                                  y_axis_limits=2500,
                                  order="tail",
                                  ylabel="Disponibilité alimentaire (Kcal/p
                                  title="Top 10 des pays avec une disponibi
```



In [164…

```
# Affichage des 10 pays qui ont le plus de disponibilité alimentaire par personne

plot_countries_by_level_indicator(data_frame=dispo_alim_zone,
```

```
                                                    column_name="dispo_al_totale",
                                                    n_countries=10,
                                                    y_axis_limits=4000,
                                                    order="head",
                                                    ylabel="Disponibilité alimentaire (Kcal/p
                                                    title="Top 10 des pays avec une disponibi
```



2.4 - Analyse de la disponibilité alimentaire intérieure

In [167…

```
# Chargement de la table disponibilité alimentaire
df_dispo_al = pd.read_csv("df_disponibilite_alimentaire.csv")


# Définition de la liste des variables à aggréger

variables_aggr = ["dispo_int", "aliments_animaux", "nourriture", "pertes",
                  "autres_ut", "exportations", "importations", "production",
                  "semences", "traitement","variation_stock"]

dispo_alimentaire = df_dispo_al.groupby("zone")[variables_aggr].sum().reset_index()
```

In [169…

```
# Calcul de la disponibilité totale /  disponibilités totales =  Production + impor
# Toutes les quantités sont en millions de tonnes
dispo_alimentaire["dispo_totale"] = dispo_alimentaire["production"] + dispo_aliment

dispo_alimentaire.head()
```

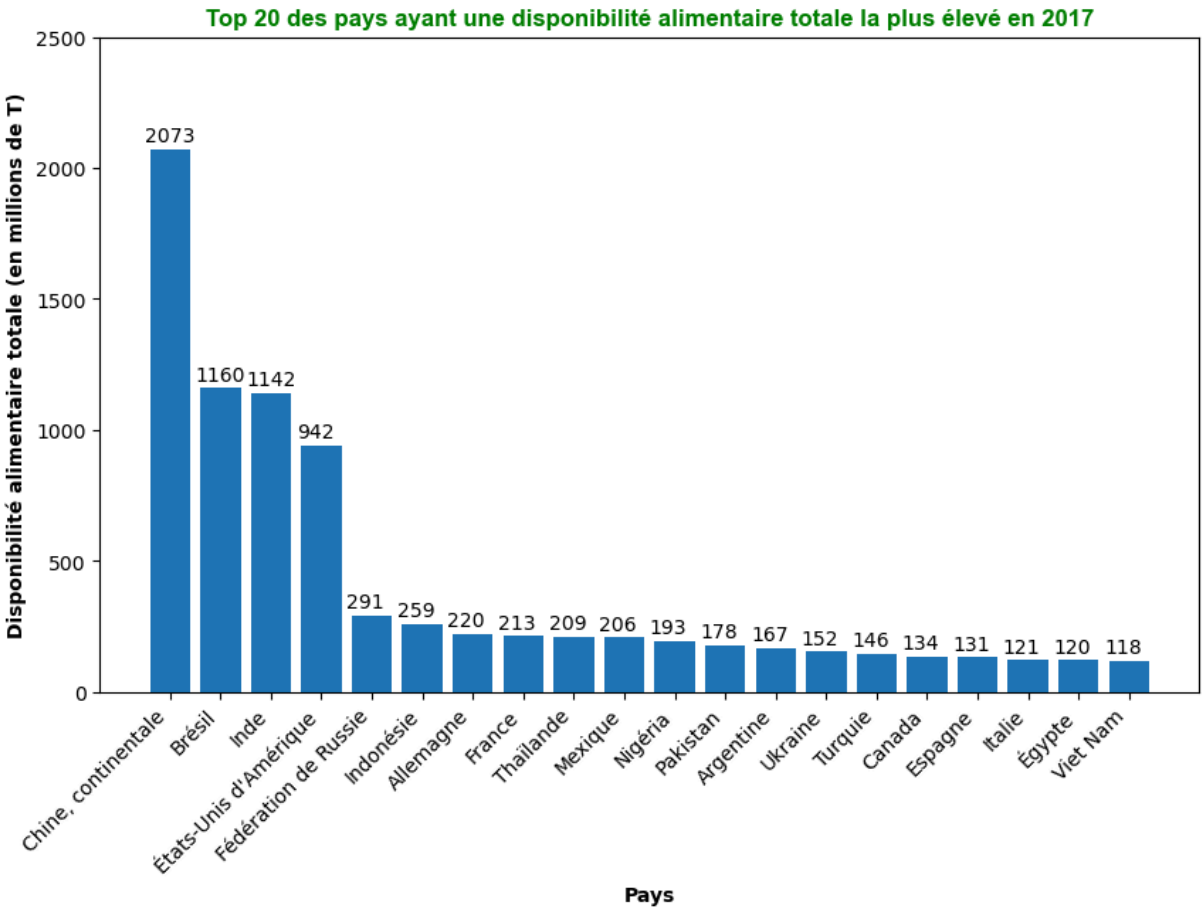
Out[169…

	zone	dispo_int	aliments_animaux	nourriture	pertes	autres_ut	exportations	in
0	Afghanistan	13.515	0.768	10.735	1.135	0.415	0.278	
1	Afrique du Sud	61.256	5.309	29.812	2.193	0.876	11.674	
2	Albanie	4.758	0.660	3.476	0.276	0.174	0.094	
3	Algérie	42.630	4.352	31.729	3.753	1.707	0.655	
4	Allemagne	162.275	30.209	79.238	3.781	7.711	57.431	

a) - Calcul du stock alimentaire total en 2017 dans chaque pays

In [172…

```
# Les 20 pays ayant les stock d'aliments les plus élevé en 2017

data = dispo_alimentaire.sort_values("dispo_totale", ascending=False).head(20)

plt.figure(figsize=(10,6))
plt.bar(height=data["dispo_totale"], x=data["zone"])

plt.ylim(0, 2500) # Pour préciser l'étendu des valeurs de l'axes des ordonnées

plt.title("Top 20 des pays ayant une disponibilité alimentaire totale la plus élevé
          color="green", fontweight="bold", fontname="Arial", fontsize=12)
plt.xlabel("Pays", fontweight ="bold")
plt.ylabel("Disponibilité alimentaire totale (en millions de T)", fontweight ="bold
plt.xticks(rotation=45, ha='right')  # ha='right' = alignement à droite

# Afficher les valeurs associées à chaque niveau

val_niv = data["dispo_totale"].tolist()
for i in range(20):
    plt.text(i-0.5, val_niv[i]+30, round(val_niv[i]))
plt.show()
```



b) - Distribution de l'utilisation de la dispobinilité alimentaire

i - Distribution au niveau mondiale

In [176…

```
# Repartition de la disponibilité alimentaire au niveau mondiale

categorie_usage = ["aliments_animaux", "nourriture", "pertes", "autres_ut", "semenc
dispo_al_usage = dispo_alimentaire[["zone", "dispo_int", "aliments_animaux", "nourr
```

In [178…

```
dispo_al_usage_mondiale = dispo_al_usage[categorie_usage].sum()
```

In [180…

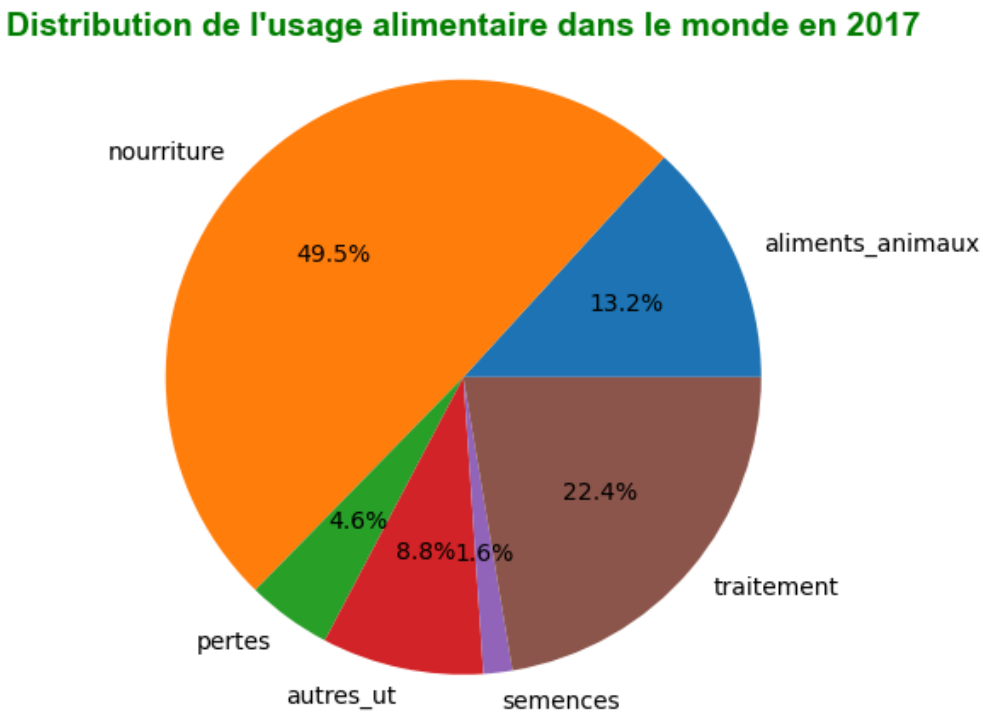
```
data = dispo_al_usage_mondiale.reset_index()
```

In [182…

```
plt.figure(figsize=(9, 5))
plt.pie(x=data[0], labels=data['index'], autopct='%.1f%%')
plt.title("Distribution de l'usage alimentaire dans le monde en 2017", color="green
plt.axis('equal')  # Equal aspect ratio ensures that pie is drawn as a circle
```

Out[182…

```
(-1.0999999699284, 1.099999998568019, -1.0999999693386537, 1.099999827619138)
```



i - Distribution au niveau de chaque pays

In [185…

```
# Calcul de la proportion de la disponibilité intérieure allouée à chaque usage.

dispo_al_usage_propor = dispo_al_usage[["zone", "dispo_int"]].copy()


# Création des variables qui indique la proportion de la disponbilité intérieure al
for categorie in  categorie_usage:
    dispo_al_usage_propor[categorie] = round((dispo_al_usage[categorie] / dispo_al_


# affichage de la table

dispo_al_usage_propor.head()
```

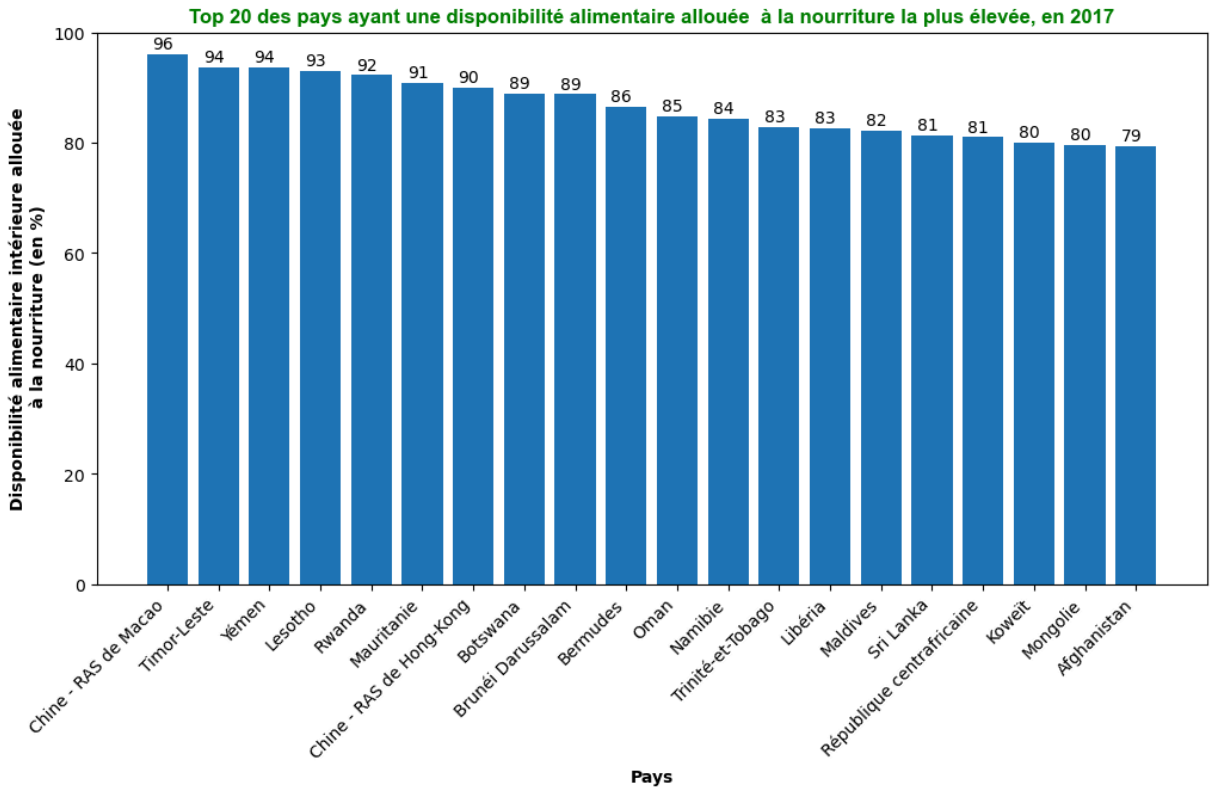
Out[185…

	zone	dispo_int	aliments_animaux	nourriture	pertes	autres_ut	semences	traite
0	Afghanistan	13.515	5.68	79.43	8.40	3.07	2.92	
1	Afrique du Sud	61.256	8.67	48.67	3.58	1.43	0.41	
2	Albanie	4.758	13.87	73.06	5.80	3.66	1.16	
3	Algérie	42.630	10.21	74.43	8.80	4.00	1.05	
4	Allemagne	162.275	18.62	48.83	2.33	4.75	0.96	

In [187…

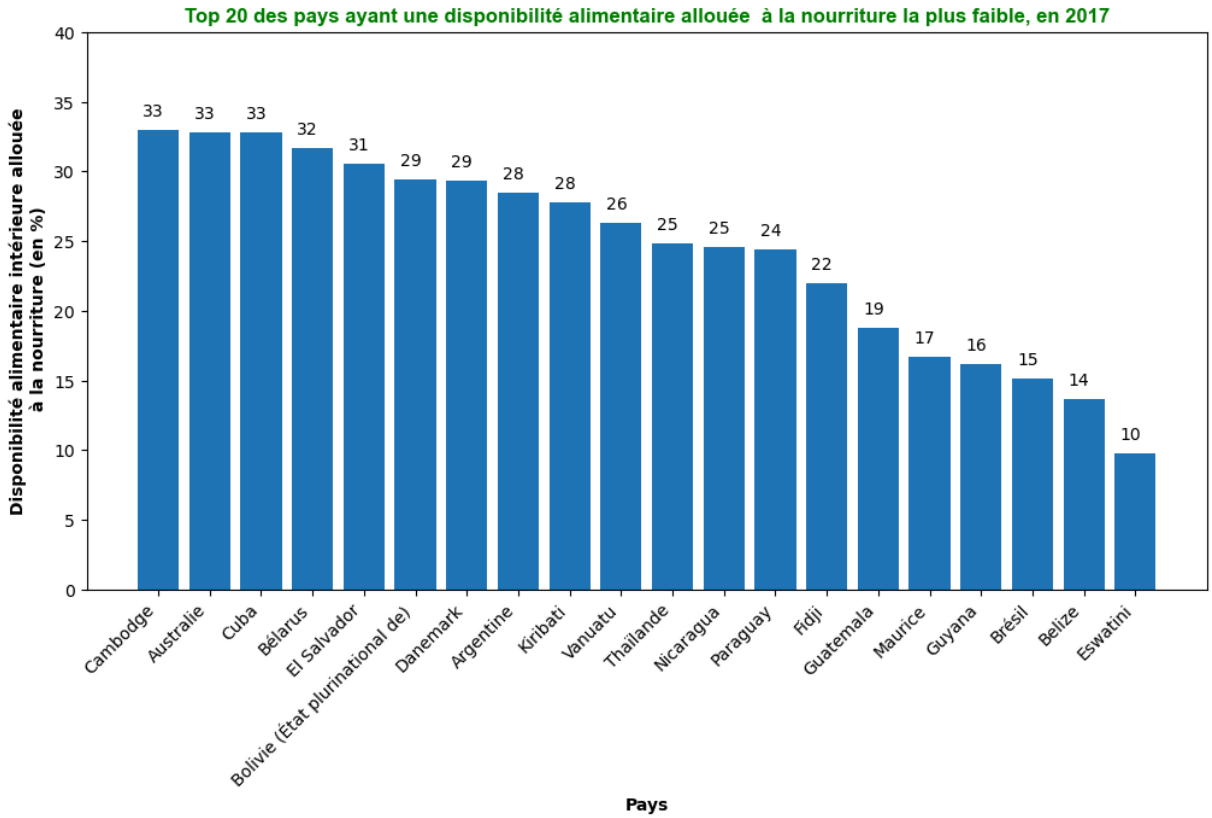
```
# Affichage du 10 des pays qui conncentre le plus de la disponibilité alimetaire po
```

```
plot_countries_by_level_indicator(data_frame=dispo_al_usage_propor,
                                  column_name="nourriture",
                                  n_countries=20,
                                  y_axis_limits=100,
                                  order="head",
                                  ylabel= "Disponibilité alimentaire intéri
                                  title="Top 20 des pays ayant une disponib
```



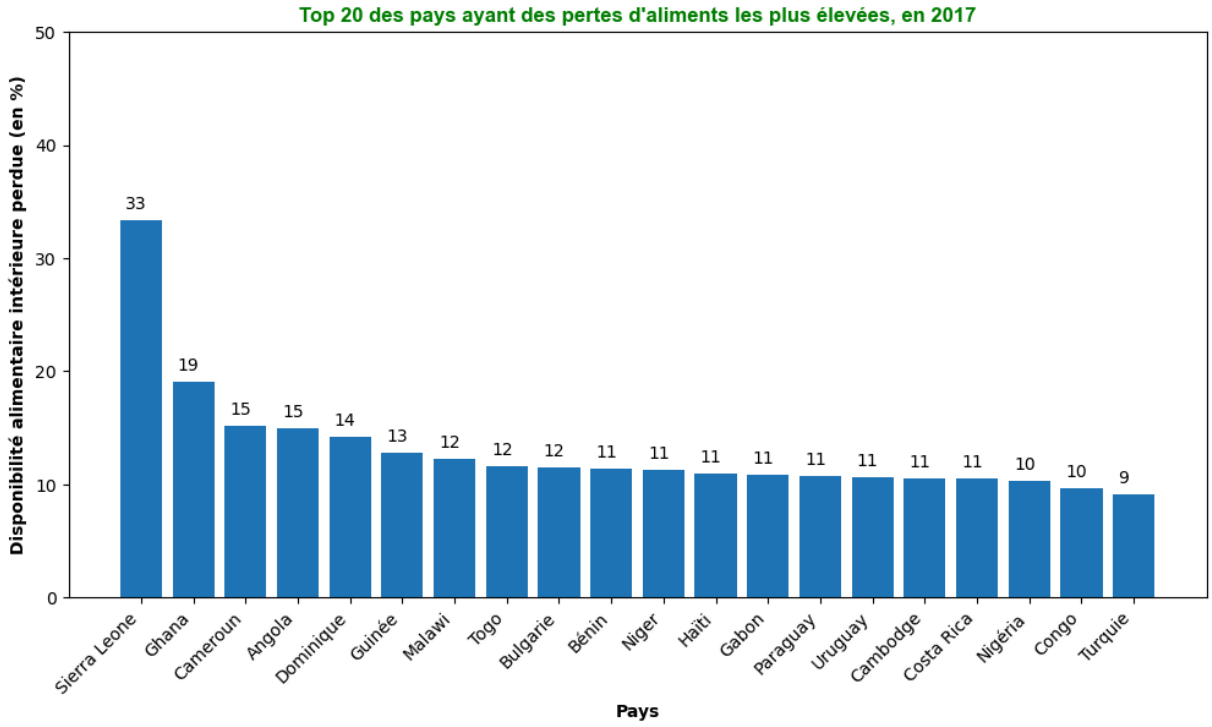
In [188…

```
# Pays qui allouent le moins à la nourriture
plot_countries_by_level_indicator(data_frame=dispo_al_usage_propor,
                                  column_name="nourriture",
                                  n_countries=20,
                                  y_axis_limits=40,
                                  order="tail",
                                  ylabel= "Disponibilité alimentaire intéri
                                  title="Top 20 des pays ayant une disponib
```



In [191…

```
# Pays ayant les les pertes les plus élevées

plot_countries_by_level_indicator(data_frame=dispo_al_usage_propor,
                                  column_name="pertes",
                                  n_countries=20,
                                  y_axis_limits=50,
                                  order="head",
                                  ylabel= "Disponibilité alimentaire intéri
                                  title="Top 20 des pays ayant des pertes d
```



2.5 - Analyse de l'aide alimentaire

In [194…

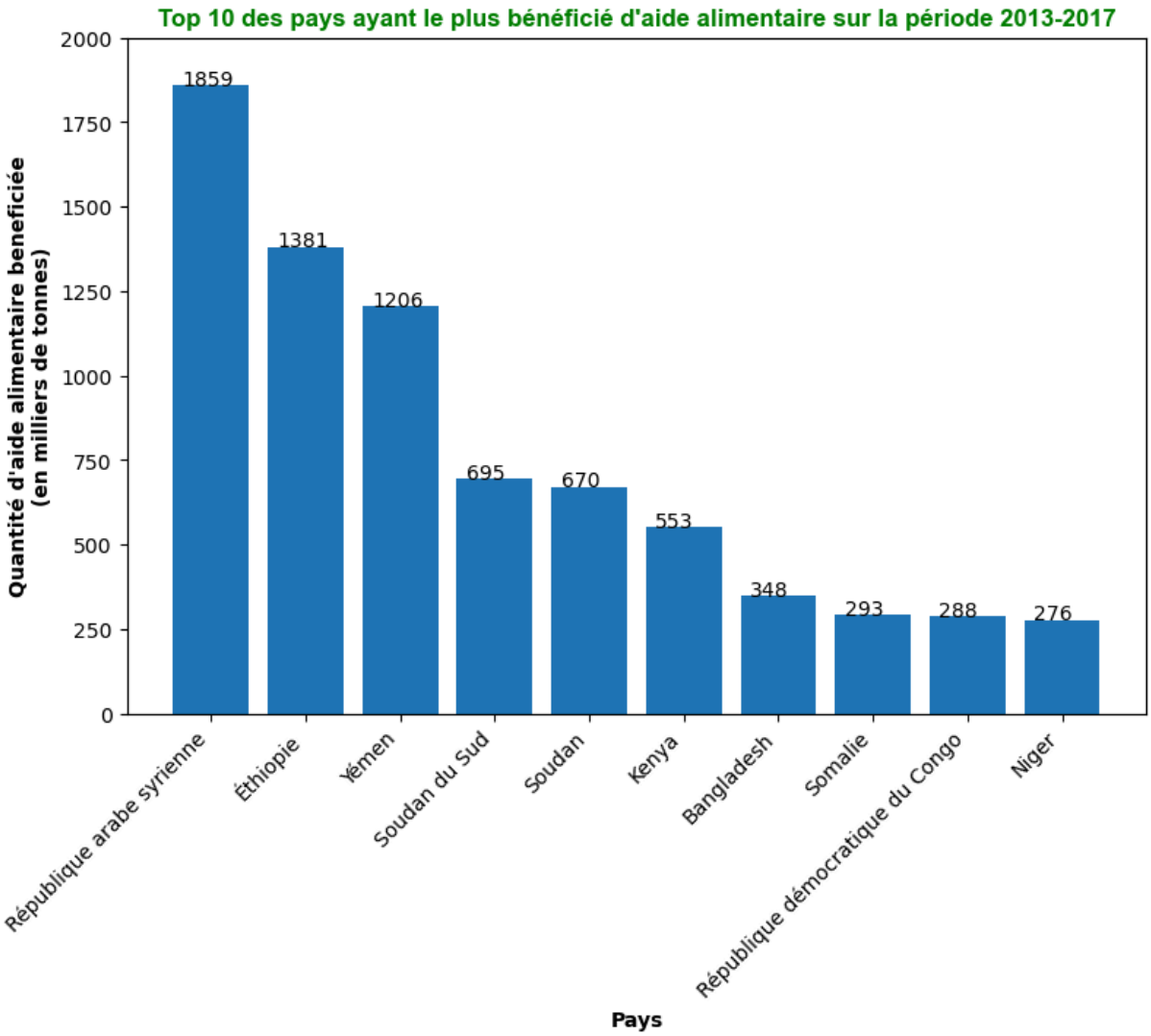
```
# Chargement des tables

# On analyse l'évolution de l'aide sur la période 2013-2017
# et donc les données sur 2017 sont supprimées

df_aide_alim = pd.read_csv("df_aide_alimentaire.csv")
df_aide_alim = df_aide_alim.loc[df_aide_alim["année"] != 2017, :]


# Calcul du total d'aide alimentaire par pays depuis sur la période 2013-2016
df_aide_alim_pays = df_aide_alim.groupby("zone")["quantité"].sum().reset_index()


# Transformation des quantités d'aide en milliers de tonnes pour faciliter la lisib
df_aide_alim_pays["quantité(en milliers T)"] = round((df_aide_alim_pays["quantité"]

df_aide_alim_pays.sort_values("quantité(en milliers T)", ascending=False).head()
```

Out[194…

	zone	quantité	quantité(en milliers T)
51	République arabe syrienne	1858943	1858.9
75	Éthiopie	1381294	1381.3
70	Yémen	1206484	1206.5
61	Soudan du Sud	695248	695.2
60	Soudan	669784	669.8

In [196…

```
# Affichage des 10 pays qui ont beneficié le plus d'aide alimentaire depuis 2013


plot_countries_by_level_indicator(data_frame=df_aide_alim_pays,
                                          column_name="quantité(en milliers T)",
                                          n_countries=10,
                                          y_axis_limits=2000,
                                          fig_dimension=(9, 6),
                                          order="head",
                                          ylabel= "Quantité d'aide alimentaire bene
                                          title="Top 10 des pays ayant le plus béné
```



In [198…

```
df_aide_alim
```

Out[198…

	zone	année	produit	quantité
0	Afghanistan	2013	Autres non-céréales	682
1	Afghanistan	2014	Autres non-céréales	335
2	Afghanistan	2013	Blé et Farin	39224
3	Afghanistan	2014	Blé et Farin	15160
4	Afghanistan	2013	Céréales	40504
...	...	...	...	...
1470	Zimbabwe	2015	Mélanges et préparations	96
1471	Zimbabwe	2013	Non-céréales	5022
1472	Zimbabwe	2014	Non-céréales	2310
1473	Zimbabwe	2015	Non-céréales	306
1474	Zimbabwe	2013	Riz, total	64

1475 rows × 4 columns

In [200…

```
# Calcul du total d'aide alimentaire par pays par année
df_aide_alim_pays_annee = df_aide_alim.groupby(["zone", "année"])["quantité"].sum()



# Liste des 5 pays qui ont le plus beneficié d'aide
liste_top5_aide = df_aide_alim_pays.sort_values("quantité(en milliers T)", ascendin


# Création d'un dataframe des 5 pays qui ont le plus beneficié de l'aide
df_aide_alim_top_5 = df_aide_alim_pays_annee.loc[df_aide_alim_pays_annee["zone"].is

# Transformation des quantités d'aide alimentaire des Tonnes en milliers de tonnes
df_aide_alim_top_5["quantité(en milliers T)"] = round((df_aide_alim_top_5["quantité
```

```
C:\Users\Fujitsu\AppData\Local\Temp\ipykernel_15868\3876394802.py:14: SettingWithCop
yWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/u
ser_guide/indexing.html#returning-a-view-versus-a-copy
  df_aide_alim_top_5["quantité(en milliers T)"] = round((df_aide_alim_top_5["quantit
é"] / 1_000), 1)
```

In [202…

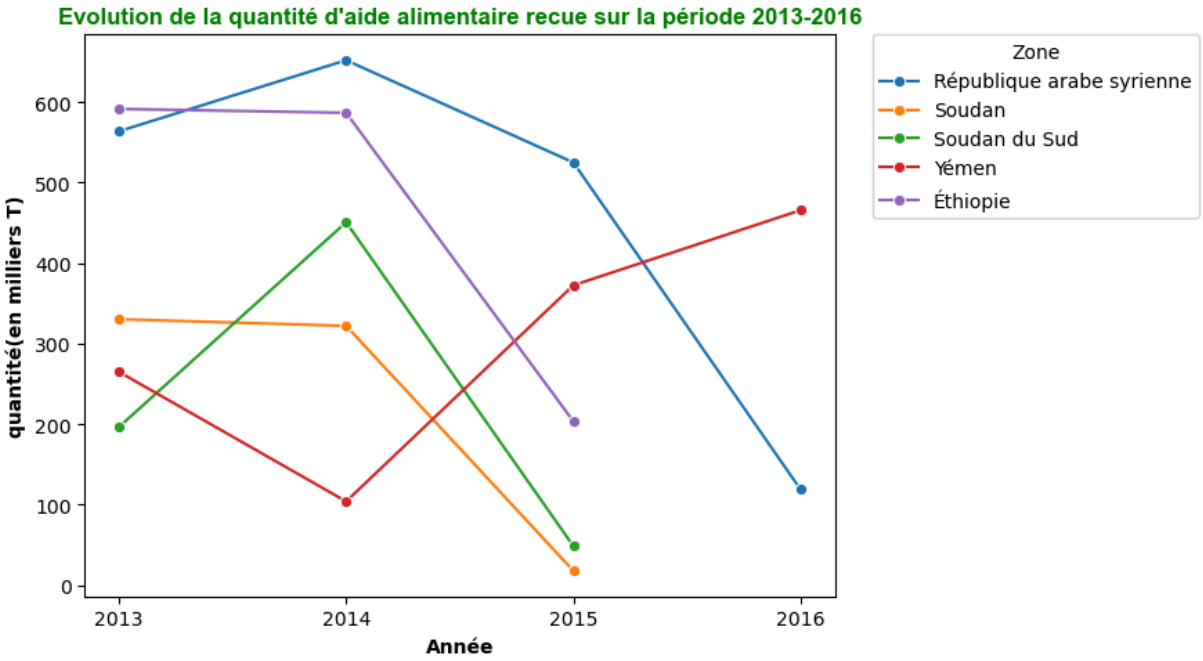
```
df_aide_alim_top_5.head()
```

Out[202…

	zone	année	quantité	quantité(en milliers T)
161	République arabe syrienne	2013	563566	563.6
162	République arabe syrienne	2014	651870	651.9
163	République arabe syrienne	2015	524949	524.9
164	République arabe syrienne	2016	118558	118.6
189	Soudan	2013	330230	330.2

In [204…

```
import matplotlib.pyplot as plt
import seaborn as sns

# Exemple de DataFrame déjà formaté
# sns.lineplot(data=df, x="année", y="quantité", hue="zone")


# Affichage de l'aide ailementaire des 5 pays par années
plt.figure(figsize=(9, 5))
sns.lineplot(data=df_aide_alim_top_5, x="année", y="quantité(en milliers T)", hue="

# Légende en dehors à droite
plt.legend(title='Zone', bbox_to_anchor=(1.05, 1), loc='upper left', borderaxespad=

# Titre et labels
plt.title("Evolution de la quantité d'aide alimentaire recue sur la période 2013-20
          color="green", fontweight="bold", fontname="Arial", fontsize=12)
plt.xlabel("Année", fontweight="bold")
plt.ylabel("quantité(en milliers T)", fontweight="bold")
plt.xticks([2013, 2014, 2015, 2016])

plt.tight_layout()  # Pour éviter que la légende dépasse de la figure
plt.show()
```



2.6 - Utilisation des céréales

a) Proportion des céréales

In [208…

```
# Chargement de la table disponibilité alimentaire
dispo_al = pd.read_csv("df_disponibilite_alimentaire.csv")


# Liste de tous les produits
produits = dispo_al["produit"].unique().tolist()

# Listes des produits d'origine animale
produits_animale = dispo_al.loc[dispo_al["origine"] == "animale", "produit"].unique


# Liste des produits d'origine végétale
produits_vegetale = dispo_al.loc[dispo_al["origine"] == "vegetale", "produit"].uniq

# Listes des céréales considérées
cereales = ['Blé', 'Maïs', 'Millet', 'Orge', 'Riz (Eq Blanchi)', 'Avoine', 'Seigle'


# Création d'une variable qui indique si le produit est une céréale ou pas
dispo_alimentaire = dispo_al.copy()

dispo_alimentaire["type_produit"] = "autre produit"
dispo_alimentaire.loc[dispo_alimentaire["produit"].isin(cereales), "type_produit"]

# Affichage du dataset
dispo_alimentaire.head()
```

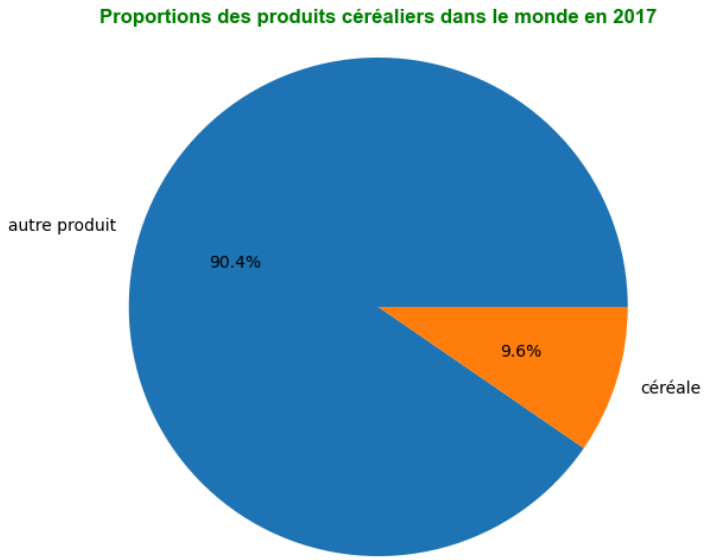
Out[208…

	zone	produit	origine	aliments_animaux	autres_ut	dispo_alim_kcal	dispo_ali
0	Afghanistan	Abats Comestible	animale	0.0	0.0	5.0	
1	Afghanistan	Agrumes, Autres	vegetale	0.0	0.0	1.0	
2	Afghanistan	Aliments pour enfants	vegetale	0.0	0.0	1.0	
3	Afghanistan	Ananas	vegetale	0.0	0.0	0.0	
4	Afghanistan	Bananes	vegetale	0.0	0.0	4.0	

In [210…

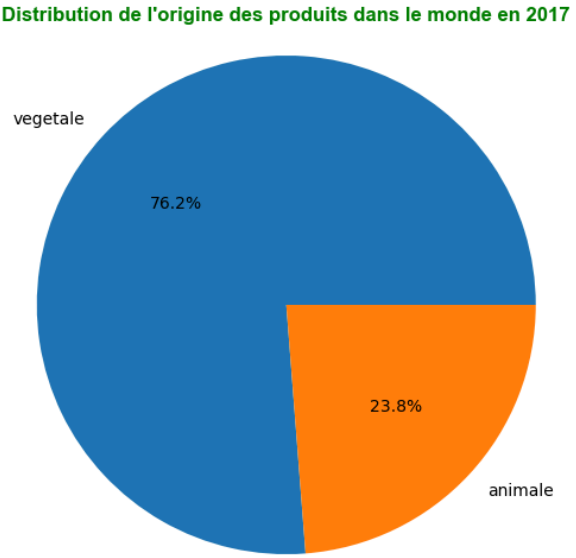
```
# Affichage de la proportion des produits céréaliers

plot_pie_chart(
```

```
    data=dispo_alimentaire,
    category_column="type_produit",
    title="Proportions des produits céréaliers dans le monde en 2017"
)
```



In [212…

```
# Affichage de la distribution de l'origine des produites

plot_pie_chart(
    data=dispo_alimentaire,
    category_column="origine",
    title="Distribution de l'origine des produits dans le monde en 2017"
)
```



b) Distribution de l'utilisation des céréales

In [215…

```
# Calcul de la quantité de céréales allouée à chaque type d'usage.
df_usage = dispo_alimentaire.groupby("type_produit")[["aliments_animaux", "nourritu
```

```
df_usage
```

Out[215…

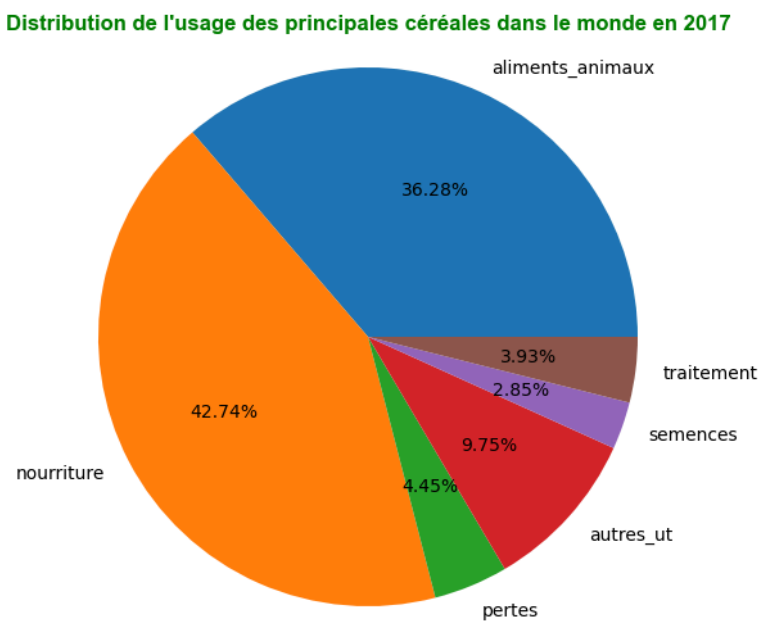
	type_produit	aliments_animaux	nourriture	traitement	semences	pertes	autres_ut
0	autre produit	430.710	3847.248	2110.098	86.143	346.578	630.236
1	céréale	873.535	1029.010	94.589	68.538	107.120	234.787

In [217…

```
# Création d'une table sur l'usage des céréales
data_ = {
    'aliments_animaux': 873.535,
    'nourriture': 1029.010,
    'pertes': 107.120,
    'autres_ut': 234.787,
    'semences': 68.538,
    'traitement': 94.589
}

# Transformation en DataFrame
data_cereales = pd.DataFrame(list(data_.items()), columns=['Utilisation', 'Quantité

# Affichage du graphique
plt.figure(figsize=(12,6))
plt.pie(x=data_cereales['Quantité'], labels=data_cereales['Utilisation'], autopct='
plt.title("Distribution de l'usage des principales céréales dans le monde en 2017",
plt.axis('equal')  # Equal aspect ratio ensures that pie is drawn as a circle
```

Out[217…

```
(-1.099993639802822,
 1.0999996971334678,
 -1.0999900802004643,
 1.099997760905171)
```



2.7 - Exemple de la Thailande pour le manioc

In [220…

```
# Création des dataframe uniquement pour la thailande

# Importation de la table sur la sous nutrition créé à la section 2.2 b
df_sous_nutrition = pd.read_csv("df_sous_nutrition_2.csv")
sous_nutrit_thai = df_sous_nutrition.loc[df_sous_nutrition["zone"] == "Thaïlande",

# table sur la disponibilité alimentaire en thailande
dispo_alim_thai = df_dispo_al.loc[df_dispo_al["zone"] == "Thaïlande", ["zone", "pro
```

In [222…

```
sous_nutrit_thai.head()
```

Out[222…

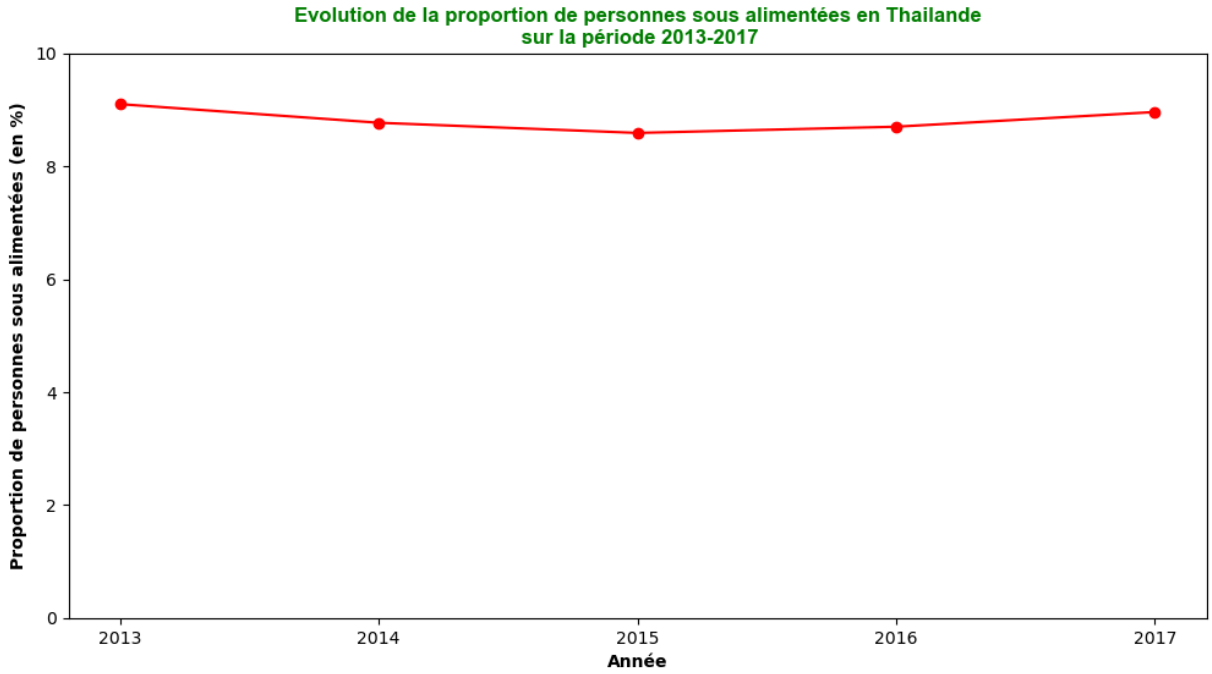
	zone	periode	année	pop_sous_alim	population	prop_pop_sous_al
925	Thaïlande	2012-2014	2013	6200000.0	68144518.0	9.10
926	Thaïlande	2013-2015	2014	6000000.0	68438746.0	8.77
927	Thaïlande	2014-2016	2015	5900000.0	68714511.0	8.59
928	Thaïlande	2015-2017	2016	6000000.0	68971308.0	8.70
929	Thaïlande	2016-2018	2017	6200000.0	69209810.0	8.96

In [224…

```
# Evolution de la proportion des personnes sous alimentées en thaillande

plt.figure(figsize=(12,6))
plt.plot(sous_nutrit_thai["année"], sous_nutrit_thai["prop_pop_sous_al"], marker='o
plt.title("Evolution de la proportion de personnes sous alimentées en Thailande\n s
          color="green", fontweight="bold", fontname="Arial", fontsize=12)
plt.xlabel("Année", fontweight ="bold")
plt.ylabel("Proportion de personnes sous alimentées (en %)", fontweight ="bold")
plt.ylim(0, 10),
plt.xticks([2013, 2014, 2015, 2016, 2017])
```

Out[224…

```
([<matplotlib.axis.XTick at 0x1c231141b50>,
  <matplotlib.axis.XTick at 0x1c231141130>,
  <matplotlib.axis.XTick at 0x1c2301c5550>,
  <matplotlib.axis.XTick at 0x1c231232a80>,
  <matplotlib.axis.XTick at 0x1c2312336e0>],
 [Text(2013, 0, '2013'),
  Text(2014, 0, '2014'),
  Text(2015, 0, '2015'),
  Text(2016, 0, '2016'),
  Text(2017, 0, '2017')])
```



In [226…

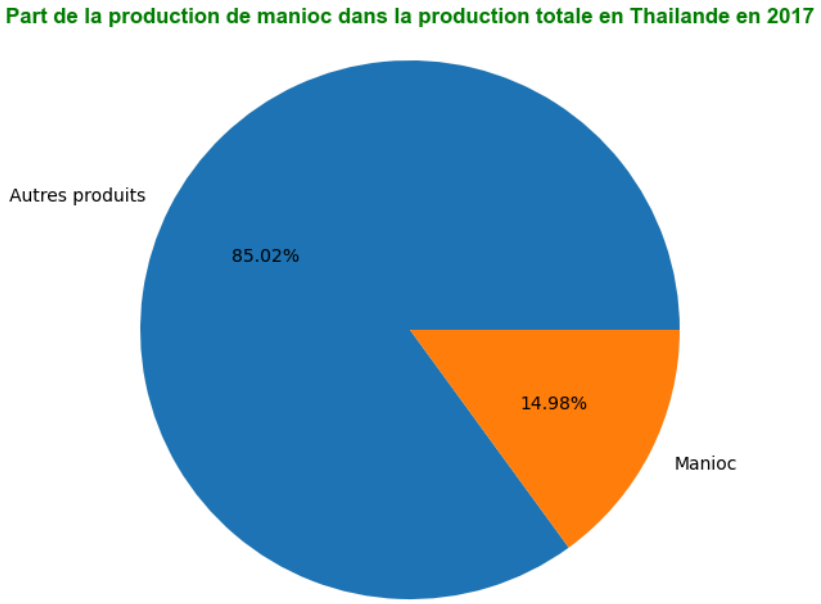
```
# Création de la variable qui indique si le produit est le manioc ou pas

dispo_alim_thai["type_produit"] = "Manioc"
dispo_alim_thai.loc[dispo_alim_thai["produit"] != "Manioc",  "type_produit"] = "Aut
```

In [228…

```
# Proportion du manioc dans la production

df = dispo_alim_thai.groupby("type_produit")["production"].sum().reset_index()

plt.figure(figsize=(12,6))
plt.pie(x=df['production'], labels=df['type_produit'], autopct='%.2f%%')
plt.title("Part de la production de manioc dans la production totale en Thailande e
          color="green", fontweight="bold", fontname="Arial", fontsize=12)
plt.axis('equal')  # Equal aspect ratio ensures that pie is drawn as a circle
```

Out[228…

```
(-1.0999998387551844,
 1.0999999923216754,
 -1.0999999832492628,
 1.0999998969288187)
```



In [230…

```
dispo_alim_thai
```

Out[230…

	zone	produit	dispo_int	exportations	importations	production	type_prod
13759	Thaïlande	Abats Comestible	0.074	0.005	0.033	0.045	Aut produ
13760	Thaïlande	Agrumes, Autres	0.008	0.006	0.002	0.012	Aut produ
13761	Thaïlande	Alcool, non Comestible	0.358	0.110	0.021	0.447	Aut produ
13762	Thaïlande	Aliments pour enfants	0.012	0.007	0.019	0.000	Aut produ
13763	Thaïlande	Ananas	0.782	1.449	0.009	2.209	Aut produ
...	...	...	...	...	...	...	
13849	Thaïlande	Viande de Suides	0.871	0.022	0.001	0.891	Aut produ
13850	Thaïlande	Viande de Volailles	0.945	0.536	0.011	1.470	Aut produ
13851	Thaïlande	Viande, Autre	-0.092	0.096	0.004	0.000	Aut produ
13852	Thaïlande	Vin	0.008	0.008	0.016	0.000	Aut produ
13853	Thaïlande	Épices, Autres	0.114	0.042	0.013	0.143	Aut produ

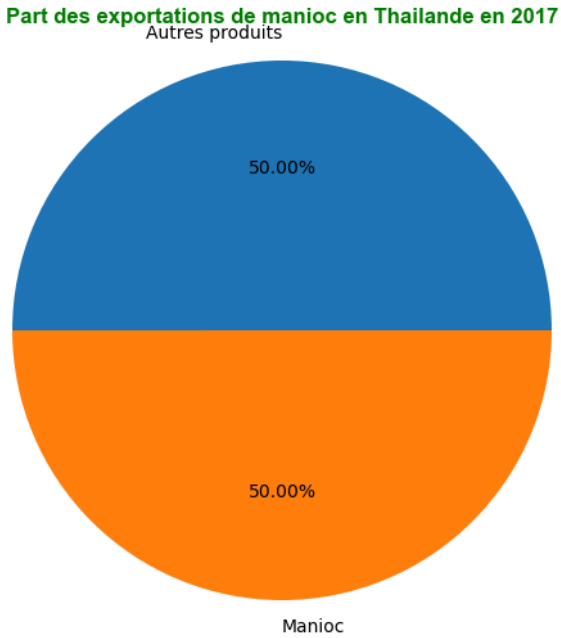
95 rows × 7 columns

In [232…

```
# Proportion du manioc dans les exportations

df2 = dispo_alim_thai.groupby("type_produit")["exportations"].sum().reset_index()

plt.figure(figsize=(12,6))
plt.pie(x=df2['exportations'], labels=df2['type_produit'], autopct='%.2f%%')
plt.title("Part des exportations de manioc en Thailande en 2017",
          color="green", fontweight="bold", fontname="Arial", fontsize=12)
plt.axis('equal')  # Equal aspect ratio ensures that pie is drawn as a circle
```

Out[232…

```
(-1.099999999999999,
 1.0999999999999999,
 -1.099999999999999,
 1.0999999999999999)
```



In [234…

```
# Table sur l'usage du manioc en Thailande

dispo_usage_manioc_thai = df_dispo_al.loc[(df_dispo_al["zone"] == "Thaïlande") & (d
                                ["zone", "produit", "dispo_int", "aliments_animaux"
                                 "semences", "traitement", "importations",      "pr
categorie_usage = ["aliments_animaux", "nourriture", "pertes", "autres_ut", "semenc
```

In [236…

```
# Les différentes usage du manioc en thailande en 2017

dispo_usage_manioc_thai
```

Out[236…

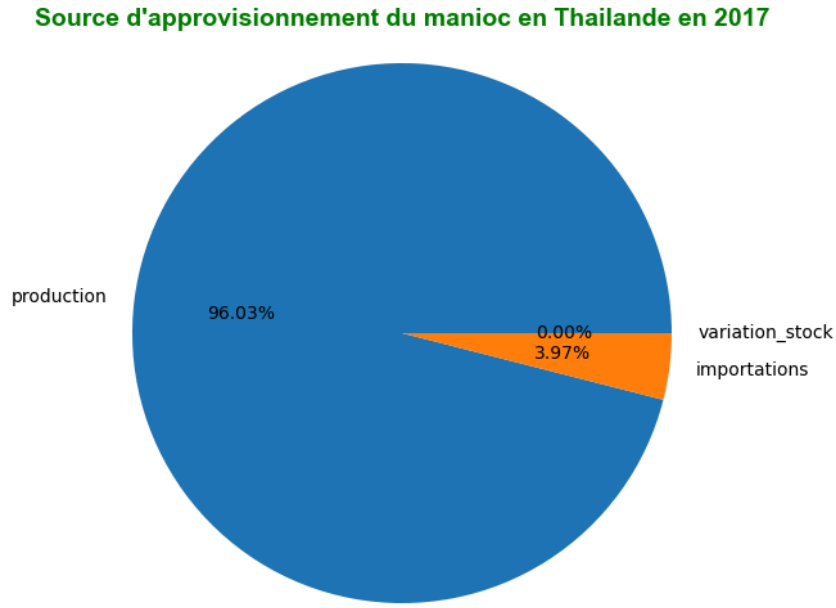
	zone	produit	dispo_int	aliments_animaux	nourriture	pertes	autres_ut	sem
13809	Thaïlande	Manioc	6.264	1.8	0.871	1.511	2.081	

In [238…

```
data2 = {
    'production': 30.228,
    'importations': 1.25,
    'variation_stock': 0.00
}

# Transformation en DataFrame
thai_approv_manioc = pd.DataFrame(list(data2.items()), columns=['provenance', 'Quan



plt.figure(figsize=(12,6))
plt.pie(x=thai_approv_manioc['Quantité'], labels=thai_approv_manioc['provenance'],
plt.title("Source d'approvisionnement du manioc en Thailande en 2017",
          color="green", fontweight="bold", fontname="Arial", fontsize=14)
plt.axis('equal')  # Equal aspect ratio ensures that pie is drawn as a circle
```

Out[238…

```
(-1.099999743180557,
 1.0999999877705027,
 -1.0999996268605585,
 1.0999999202796782)
```

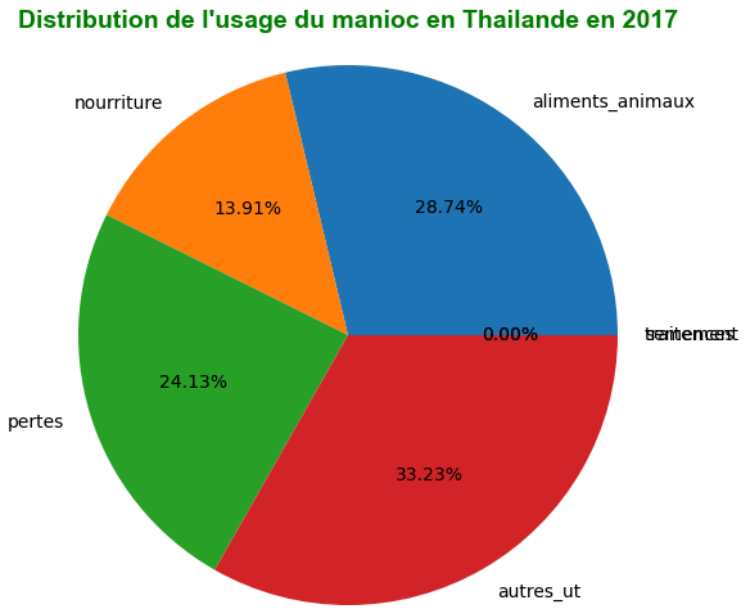


In [240…

```
data3 = {
    'aliments_animaux': 1.8,
    'nourriture': 0.871,
    'pertes': 1.511,
    'autres_ut': 2.081,
    'semences': 0.0,
    'traitement': 0.0
}

# Transformation en DataFrame
thai_usage_manioc = pd.DataFrame(list(data3.items()), columns=['Utilisation', 'Quan


plt.figure(figsize=(12,6))
plt.pie(x=thai_usage_manioc['Quantité'], labels=thai_usage_manioc['Utilisation'], a
plt.title("Distribution de l'usage du manioc en Thailande en 2017",
          color="green", fontweight="bold", fontname="Arial", fontsize=14)
plt.axis('equal')  # Equal aspect ratio ensures that pie is drawn as a circle
```

Out[240…

```
(-1.099978701719947, 1.099998985796188, -1.099999948125126, 1.0999989135998751)
```



In [242…

```
# Exportation et production de manioc en thaillande

dispo_alim_thai.loc[dispo_alim_thai["produit"] == "Manioc",  ["zone", "exportations
```

Out[242…

	zone	exportations	production
13809	Thaïlande	25.214	30.228